JavaScript cheatsheet · 19 chapters · only JavaScript code

Playwright with JavaScript: cheatsheet

The key ideas, every example program in JavaScript, the tables and the common mistakes from every chapter, in one place for quick revision. Everything here comes from the chapters and shows JavaScript only, so this sheet is all you need. Each card links back to the lesson that teaches it.

95 key ideas | 101 code examples | 78 tables | 74 tips and traps

STEP 1 Getting started

1.1 What Playwright is and how it works

What Playwright is and the jobs teams use it for, the official languages, a fact based comparison with Selenium and Cypress, a first script in four languages, how one call travels through the client library, the driver and the browser protocol with real DEBUG=pw:api, pw:channel and pw:protocol logs, a first test with JUnit, pytest and Playwright Test, a recorded sign in walk, and what a test runner does for every test.

KEY IDEAS

- **You write steps** Open, type, click, check.
- **Messages** Each call becomes a small message. The driver is the same for every language.
- **Real browsers** Real clicks and real typing. The page cannot tell it apart from a person.
- **It waits** Visible, stable, enabled. No fixed sleeps, so far fewer flaky tests.
- **Remember** Playwright drives real browsers through one driver, waits for elements before it acts, and works the same way in Java, Python, JavaScript and TypeScript.

CODE

Your first Playwright script

first.mjs

```
import { chromium } from 'playwright';

const browser = await chromium.launch();          // start a real Chromium, with no window
const page = await browser.newPage();             // open a new tab
await page.goto('http://localhost:4173/products');
console.log('Title:', await page.title());
await page.screenshot({ path: 'products.png' });  // save a picture of the page
await browser.close();
```

Run it: `node first.mjs && ls *.png`

Lesson: Your first script

Watch what Playwright does inside each call

first.mjs

```
import { chromium } from 'playwright';

const browser = await chromium.launch();
const page = await browser.newPage();
await page.goto('http://localhost:4173/login');
await page.getByLabel('Email').fill('asha@shop.example');
await page.getByRole('button', { name: 'Sign in' }).click();
await browser.close();
```

Run it: `DEBUG=pw:api node first.mjs`

Lesson: How one line travels to the browser

The messages your code sends to the driver

first.mjs

```
import { chromium } from 'playwright';

const browser = await chromium.launch();
const page = await browser.newPage();
await page.goto('http://localhost:4173/login');
await page.getByLabel('Email').fill('asha@shop.example');
await page.getByRole('button', { name: 'Sign in' }).click();
await browser.close();
```

Run it: `DEBUG=pw:channel node first.mjs`

Lesson: How one line travels to the browser

The messages the driver sends to Chromium

first.mjs

```
import { chromium } from 'playwright';

const browser = await chromium.launch();
const page = await browser.newPage();
await page.goto('http://localhost:4173/login');
await page.getByLabel('Email').fill('asha@shop.example');
await page.getByRole('button', { name: 'Sign in' }).click();
await browser.close();
```

Run it: `DEBUG=pw:protocol node first.mjs`

Lesson: How one line travels to the browser

Your first test, in four languages

tests/sign-in.spec.js

```
import { test, expect } from '@playwright/test';

test('a customer can sign in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: From a script to a test

A customer signs in

`await page.goto('http://localhost:4173/login');`	Open the sign in page.
`await page.getByLabel('Email').fill('asha@shop.example');`	Type the email into the box labelled Email.
`await page.getByLabel('Password').fill('s3cret');`	Type the password.
`await page.getByRole('button', { name: 'Sign in' }).click();`	Click the button called Sign in. The shop signs Asha in and opens the products page.
`await expect(page.getByText('Hello, Asha')).toBeVisible();`	Check that the greeting says Hello, Asha. The check waits until it appears.

Lesson: From a script to a test

TABLES

Table: language · package · usual test runner · version used on this page

Language	Package	Usual test runner	Version used on this page
🟨 JavaScript and 🔷 TypeScript	`@playwright/test` on npm	Playwright Test (built in)	1.63.0

Lesson: What Playwright is

Table: playwright · selenium webdriver · cypress

	Playwright	Selenium WebDriver	Cypress
Languages	Java, Python, JavaScript, TypeScript, C#	Java, Python, JavaScript, C#, Ruby	JavaScript and TypeScript
Browsers	Chromium, Firefox, WebKit	Chrome, Firefox, Edge, Safari through their drivers	Chromium family, Firefox, WebKit (experimental)
How it talks to the browser	Browser debugging protocols, through one driver process	The W3C WebDriver protocol, through a driver per browser	Runs inside the browser next to your app
Waiting	Automatic before every action and in every check	You add waits yourself	Automatic retries for commands and checks
Several tabs, windows and domains in one test	Yes	Yes	Limited
Parallel runs	Built in, free	With Selenium Grid or a test runner	Paid cloud service, or plugins
Debugging	Trace viewer, UI mode, inspector, codegen	Logs and screenshots you add	Time travel in the runner

Lesson: Why teams pick Playwright

Table: job · 🟨 javascript and 🔷 typescript

Job	🟨 JavaScript and 🔷 TypeScript
Run the tests	`npx playwright test`
Where settings live	`playwright.config.ts`
A fresh page for each test	the `page` fixture
Checks that wait	`expect(...)`
Run in parallel	built in, one worker per CPU core by default
HTML report	built in, plus Allure

Lesson: What the test runner does for you

Table: module · you will learn

Module	You will learn
🚀 Getting started	what Playwright is, how it works inside, installing it in four languages, codegen and VS Code
🌐 Browser handling	browsers, contexts, pages, devices, tabs, pop ups, frames, dialogs, downloads and uploads
🎯 Finding and using elements	locators, actions and auto waiting, and checks that wait
🏃 The test runner	configuration, tags, data driven tests, fixtures, hooks, parallel runs, retries and timeouts
🔌 Network and API	mocking the server, recording traffic, API tests and signing in once
🐞 Debugging and quality	trace viewer, UI mode, screenshots, videos, visual and accessibility tests, the clock
🏗️ Framework design	page objects, a real company framework built step by step in each language, CI and Docker
📊 Reporting labs	the built in reporters, HTML report, Allure, pytest-html, ExtentReports and your own reporter
🆕 New and AI features	what changed from version 1.45 to 1.64, the Playwright MCP server and test agents

Lesson: What this subject covers

TIPS AND TRAPS

COMMON MISTAKE People often ask for Playwright in C, C++ or Go. There is no official Playwright for C or C++. For Go there is a community project, playwright-go, which is not made by the Playwright team. This subject sticks to the official languages so that every example is supported and up to date.

Lesson: What Playwright is

IN REAL TEAMS In many companies you will join a project that already uses Selenium. Knowing both is useful. Many teams move to Playwright one test suite at a time, because the ideas are close: find an element, act on it, check the result.

Lesson: Why teams pick Playwright

REMEMBER The driver is the same Node.js program for every language. This is why a new feature arrives in Java, Python, JavaScript and C# in the same release.

Lesson: How one line travels to the browser

REMEMBER The test asks for `{ page }`. Playwright Test sees the name, makes a fresh context and page for this test and closes them afterwards. The `baseURL` in playwright.config makes `/login` mean http://localhost:4173/login.

Lesson: From a script to a test

1.2 Installing Playwright and your first project

Runtime and operating system requirements, installing with Maven, pip and npm init playwright, checking the version, where browsers are installed and Chrome for Testing, the real files of a scaffolded project and typical Python and Java layouts, codegen with real screenshots and the recorded code in four languages, cleaning up recorded code, running, filtering and listing tests, editors and updating safely.

KEY IDEAS

- **Runtime** Install your language first. Check it with a version command.
- **Package** Pin the exact version.
- **Browsers** Playwright brings its own. --with-deps adds the Linux libraries.
- **Record** Click around, get code. Then clean it up and add a check.
- **Remember** Install the runtime, the Playwright package and its browsers, record a first draft with codegen, then turn it into a real test with a clear name and a check.

CODE

Check what is installed

Run it: `npx playwright --version`

Lesson: Install Playwright in your language

Where the browsers live

Run it: `npx playwright install --dry-run chromium`

Lesson: Install Playwright in your language

What codegen recorded, run as it is

tests/example.spec.js

```
import { test, expect } from '@playwright/test';

test.use({
  viewport: {
    height: 600,
    width: 800
  }
});

test('test', async ({ page }) => {
  await page.goto('http://localhost:4173/login');
  await page.getByRole('textbox', { name: 'Email' }).click();
  await page.getByRole('textbox', { name: 'Email' }).fill('asha@shop.example');
  await page.getByRole('textbox', { name: 'Password' }).click();
  await page.getByRole('textbox', { name: 'Password' }).fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
});
```

Run it: `npx playwright test`

Lesson: Record a test with codegen

The same test after a clean up

tests/sign-in.spec.js

```
import { test, expect } from '@playwright/test';

test('a customer can sign in', async ({ page }) => {
  await page.goto('/login');
  await page.getByRole('textbox', { name: 'Email' }).fill('asha@shop.example');
  await page.getByRole('textbox', { name: 'Password' }).fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: Record a test with codegen

Run only one test from a file

tests/shop.spec.js

```
import { test, expect } from '@playwright/test';

test('a customer can sign in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});

test('the products page has a title', async ({ page }) => {
  await page.goto('/products');
  await expect(page).toHaveTitle('Products · Brainy Shop');
});
```

Run it: `npx playwright test -g "sign in"`

Lesson: Running tests from the terminal

List the tests without running them

tests/shop.spec.js

```
import { test, expect } from '@playwright/test';

test('a customer can sign in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});

test('the products page has a title', async ({ page }) => {
  await page.goto('/products');
  await expect(page).toHaveTitle('Products · Brainy Shop');
});
```

Run it: `npx playwright test --list`

Lesson: Running tests from the terminal

TABLES

Table: 🟨 javascript and 🔷 typescript

	🟨 JavaScript and 🔷 TypeScript
Runtime	Node.js 22, 24 or 26
Package tool	npm, yarn or pnpm
Editor most teams use	VS Code with the Playwright extension
Operating system	same

Lesson: What you need before you start

Table: i want to · 🟨🔷 playwright test

I want to	🟨🔷 Playwright Test
run every test	`npx playwright test`
run one file	`npx playwright test tests/shop.spec.ts`
run tests by name	`-g "sign in"`
see the browser	`--headed`
slow every action down	`launchOptions: { slowMo: 500 }` in the config
pick a browser	`--project=firefox`
debug step by step	`--debug` or `--ui`
run 4 at a time	`--workers=4`

Lesson: Running tests from the terminal

Table: language · update the package · then

Language	Update the package	Then
🟨🔷 JavaScript and TypeScript	`npm install -D @playwright/test@latest`	`npx playwright install --with-deps`

Lesson: Updating Playwright safely

TIPS AND TRAPS

COMMON MISTAKE Keep one Playwright version per project and update on purpose. Each Playwright release expects its own browser builds. After you update the package, run the install command again, or the tests stop with an error that says the browser executable does not exist.

Lesson: Install Playwright in your language

IN REAL TEAMS Codegen picks locators the same way you should: role and accessible name first, then label, placeholder or text, and a test id only when nothing better exists. When the recorded locator looks odd, the page itself is often missing a label, which is worth reporting to the developers.

Lesson: Record a test with codegen

IN REAL TEAMS Turn on format on save and a linter in your editor from day one.

Lesson: Your editor

REMEMBER Pin exact versions in your project (no ^ in package.json, == in requirements.txt). Then a teammate and the CI server always run the same Playwright, and an update is a change you can see and review.

Lesson: Updating Playwright safely

STEP 2 Browser handling

2.1 Browsers, contexts, pages and devices

The browser, context and page levels with an interactive builder, launch options such as headless, slowMo and channel, two signed in users in one browser, context options for viewport, locale, time zone, colour scheme, touch, geolocation and permissions, device profiles, new tabs and popups, Chromium, Firefox and WebKit projects, persistent profiles, connectOverCDP and browser servers.

KEY IDEAS

- **Browser** Start once, about 0.5 s. Shared by many tests.
- **Context** A fresh, empty profile. Its own cookies, storage and permissions.
- **Page** One tab. Tabs in a context share the login.
- **Emulation** Size, locale, time zone, location. Set on the context, nothing on your computer changes.
- **Remember** Start one browser, give every test a new context, and set what the site should see, such as a phone or a German user, with context options.

CODE

Launch a browser with options

main.mjs

```
import { chromium } from 'playwright';

const browser = await chromium.launch({
  headless: true,      // no window. Use false to watch it work
  slowMo: 100,         // wait 100 ms before every action
  args: ['--lang=en-GB'],
});
console.log('Browser type:', browser.browserType().name());
console.log('Version:', browser.version());
console.log('Connected:', browser.isConnected());
console.log('Contexts so far:', browser.contexts().length);
await browser.close();
console.log('Connected after close:', browser.isConnected());
```

Run it: `node main.mjs`

Lesson: Launching a browser

Two shoppers in one browser, kept apart

tests/contexts.spec.js

```
import { test, expect } from '@playwright/test';

test('two shoppers in one browser', async ({ browser }) => {
  // Each context is a separate, empty browser profile.
  const ashaContext = await browser.newContext();
  const guestContext = await browser.newContext();
  const asha = await ashaContext.newPage();
  const guest = await guestContext.newPage();

  await asha.goto('/login');
  await asha.getByLabel('Email').fill('asha@shop.example');
  await asha.getByLabel('Password').fill('s3cret');
  await asha.getByRole('button', { name: 'Sign in' }).click();
  await expect(asha.getByText('Hello, Asha')).toBeVisible();

  await guest.goto('/products');
  await expect(guest.getByRole('link', { name: 'Sign in' })).toBeVisible();
  console.log('Asha cookies:', (await ashaContext.cookies()).length);
  console.log('Guest cookies:', (await guestContext.cookies()).length);

  await ashaContext.close();
  await guestContext.close();
});
```

Run it: `npx playwright test`

Lesson: Contexts keep tests apart

Pretend to be a German phone user in India

tests/emulation.spec.js

```
import { test, expect } from '@playwright/test';

test.use({
  viewport: { width: 390, height: 844 },
  locale: 'de-DE',
  timezoneId: 'Asia/Kolkata',
  colorScheme: 'dark',
  hasTouch: true,
});

test('the shop sees a German phone user in India', async ({ page }) => {
  await page.goto('/about-you');
  for (const id of ['width', 'lang', 'tz', 'price', 'touch']) {
    console.log(id.padEnd(6), await page.locator('#' + id).textContent());
  }
  const dark = await page.evaluate(() => matchMedia('(prefers-color-scheme: dark)').matches);
  console.log('dark  ', dark);
  await expect(page.locator('#lang')).toHaveText('de-DE');
});
```

Run it: `npx playwright test`

Lesson: Pretend to be another user

Location and permissions

tests/location.spec.js

```
import { test, expect } from '@playwright/test';

test.describe('with the location shared', () => {
  test.use({
    geolocation: { latitude: 28.6139, longitude: 77.209 },    // New Delhi
    permissions: ['geolocation'],
  });

  test('finds stores near New Delhi', async ({ page }) => {
    await page.goto('/about-you');
    await page.getByRole('button', { name: 'Find stores near me' }).click();
    await expect(page.locator('#where')).toHaveText('28.6139, 77.2090');
  });
});

test('location blocked when permission is not granted', async ({ page }) => {
  await page.goto('/about-you');
  await page.getByRole('button', { name: 'Find stores near me' }).click();
  await expect(page.locator('#where')).toContainText('blocked');
});
```

Run it: `npx playwright test`

Lesson: Pretend to be another user

Emulate a phone from the device list

tests/device.spec.js

```
import { test, expect, devices } from '@playwright/test';

test.use({ ...devices['Pixel 7'] });

test('the shop on a Pixel 7', async ({ page }) => {
  await page.goto('/about-you');
  console.log('width ', await page.locator('#width').textContent());
  console.log('touch ', await page.locator('#touch').textContent());
  console.log('mobile', await page.evaluate(() => /Mobile/.test(navigator.userAgent)));
  await expect(page.locator('#touch')).toHaveText('yes');
});
```

Run it: `npx playwright test`

Lesson: Phones and tablets

New tabs and pop up windows

tests/tabs.spec.js

```
import { test, expect } from '@playwright/test';

test('help opens in a new tab', async ({ page }) => {
  await page.goto('/widgets');
  // Start waiting for the new tab before the click that opens it.
  const popupPromise = page.waitForEvent('popup');
  await page.getByRole('link', { name: 'Open help in a new tab' }).click();
  const help = await popupPromise;

  await expect(help).toHaveTitle('Help · Brainy Shop');
  console.log('Tabs open:', page.context().pages().length);
  await help.close();
  console.log('Tabs open:', page.context().pages().length);
});

test('the chat window is a popup', async ({ page }) => {
  await page.goto('/widgets');
  const popupPromise = page.waitForEvent('popup');
  await page.getByRole('button', { name: 'Open chat window' }).click();
  const chat = await popupPromise;
  await expect(chat.getByRole('heading')).toHaveText('Chat with us');
});
```

Run it: `npx playwright test`

Lesson: New tabs and pop ups

A persistent profile that survives a restart

main.mjs

```
import { chromium } from 'playwright';
import { mkdtempSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const profile = mkdtempSync(join(tmpdir(), 'shop-profile-'));

// First visit: a real profile folder on disk, like a person's Chrome profile.
let context = await chromium.launchPersistentContext(profile);
let page = context.pages()[0];
await page.goto('http://localhost:4173/');
await page.evaluate(() => localStorage.setItem('theme', 'dark'));
await context.close();

// Second visit with the same folder: the data is still there.
context = await chromium.launchPersistentContext(profile);
page = context.pages()[0];
await page.goto('http://localhost:4173/');
console.log('theme =', await page.evaluate(() => localStorage.getItem('theme')));
await context.close();
```

Run it: `node main.mjs`

Lesson: Profiles, closing and remote browsers

TABLES

Table: option · what it does · when you use it

Option	What it does	When you use it
headless	run without a window (the default)	always on CI, turn off to watch a test locally
slowMo	pause before each action	to follow a test with your eyes or record a demo
channel	use an installed branded browser: `chrome`, `msedge`, `chrome-beta`	when the team must test the exact browser users have
args	extra command line switches for the browser	rare cases, such as a language or a fake camera
executablePath	start a browser from a path you give	company images with their own browser build
timeout	how long to wait for the browser to start	slow CI machines

Lesson: Launching a browser

Table: option · example · what the page sees

Option	Example	What the page sees
viewport	390 × 844	`innerWidth`, media queries
locale	de-DE	`navigator.language`, number and date formats
timezoneId	Asia/Kolkata	dates and times
colorScheme	dark	`prefers-color-scheme`
hasTouch, isMobile	true	touch events, mobile layout
geolocation, permissions	New Delhi	the location API
userAgent	any string	`navigator.userAgent`
offline	true	no network at all

Lesson: Pretend to be another user

Table: 🟨🔷 playwright test

	🟨🔷 Playwright Test
pick the engine	a project with `devices['Desktop Firefox']`
all three in one run	three projects in the config
branded Chrome or Edge	`channel: 'chrome'`

Lesson: Chromium, Firefox and WebKit

TIPS AND TRAPS

IN REAL TEAMS Real projects use two contexts in one test for features with two people: a chat between a customer and a support agent, an admin approving an order while the customer waits, or two players in a game. Each person gets their own context.

Lesson: Contexts keep tests apart

COMMON MISTAKE Emulation catches most layout and touch bugs and costs nothing. Some bugs only show on real hardware, such as a keyboard covering a field or a slow phone. Many teams run emulated phones on every commit and real devices in a cloud service before a release.

Lesson: Phones and tablets

REMEMBER WebKit on Windows and Linux is the engine of Safari, not Safari itself. It catches most Safari bugs. For the last few percent, teams run a small smoke suite on real Safari on a Mac.

Lesson: Chromium, Firefox and WebKit

2.2 Navigation, frames, dialogs, files and events

What goto waits for, waiting for the URL after a click, back, forward and reload, a recorded walk through a payment iframe, frameLocator and contentFrame, alert, confirm and prompt with the default dismiss, downloads with suggestedFilename and saveAs, uploads from memory, console, pageerror and request events, evaluate and addInitScript, and locating inside shadow DOM.

KEY IDEAS

- **Navigation** goto waits for load. After a click, wait for the new URL.
- **Iframes** Step in first. Then locate as usual.
- **Dialogs** Dismissed by default. Handle them before the click.
- **Files** Wait for downloads. Fill file inputs directly.
- **Remember** Start waiting before the click for anything the click creates: a new URL, a popup, a dialog or a download.

CODE

Go, wait for the address, go back and reload

tests/navigation.spec.js

```
import { test, expect } from '@playwright/test';

test('moving around the shop', async ({ page }) => {
  await page.goto('/login', { waitUntil: 'domcontentloaded' });
  console.log('1.', page.url());

  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('**/products');            // the app moves on by itself
  console.log('2.', page.url());

  await page.goBack();
  await expect(page).toHaveURL(/\/login$/);
  console.log('3.', page.url());

  await page.goForward();
  await page.reload();
  await expect(page).toHaveURL(/\/products$/);
  console.log('4.', page.url(), '|', await page.title());
});
```

Run it: `npx playwright test`

Lesson: Going to pages and waiting for them

Paying inside an iframe

`await page.goto('http://localhost:4173/frames');`	Open the checkout page. The card form is an iframe from the payment company.
`await page.frameLocator('iframe[title="Payment form"]').getByLabel('Card number').fill('4111 1111 1111 1111');`	Step into the iframe and type the card number.
`await page.frameLocator('iframe[title="Payment form"]').getByLabel('Expiry').fill('12/29');`	Type the expiry date.
`await page.frameLocator('iframe[title="Payment form"]').getByRole('button', { name: 'Pay ₹219' }).click();`	Click Pay inside the iframe.
`await expect(page.getByRole('status')).toHaveText('Paid with card ending 1111');`	Back in the main page, the shop shows the result.

Lesson: Iframes

Fill a card form that lives inside an iframe

tests/frames.spec.js

```
import { test, expect } from '@playwright/test';

test('pay inside the payment iframe', async ({ page }) => {
  await page.goto('/frames');
  const payment = page.frameLocator('iframe[title="Payment form"]');

  await payment.getByLabel('Card number').fill('4111 1111 1111 1111');
  await payment.getByLabel('Expiry').fill('12/29');
  await payment.getByRole('button', { name: 'Pay ₹219' }).click();

  // The message appears in the main page, outside the iframe.
  await expect(page.getByRole('status')).toHaveText('Paid with card ending 1111');
  console.log('Frames on the page:', page.frames().length);
});

test('a short card number is refused', async ({ page }) => {
  await page.goto('/frames');
  // Another way in: find the iframe element, then step into it.
  const payment = page.getByTitle('Payment form').contentFrame();
  await payment.getByLabel('Card number').fill('4111');
  await payment.getByRole('button', { name: 'Pay ₹219' }).click();
  await expect(payment.getByRole('alert')).toHaveText('Card number must have 16 digits');
});
```

Run it: `npx playwright test`

Lesson: Iframes

Alerts, confirms and prompts

tests/dialogs.spec.js

```
import { test, expect } from '@playwright/test';

test('with no handler, dialogs are dismissed', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Delete account' }).click();
  await expect(page.getByRole('status')).toHaveText('Kept your account');
});

test('accept a confirm and read its message', async ({ page }) => {
  await page.goto('/widgets');
  page.on('dialog', async dialog => {
    console.log(dialog.type(), '|', dialog.message());
    await dialog.accept();
  });
  await page.getByRole('button', { name: 'Delete account' }).click();
  await expect(page.getByRole('status')).toHaveText('Account deleted');
});

test('type an answer into a prompt', async ({ page }) => {
  await page.goto('/widgets');
  page.once('dialog', dialog => dialog.accept('Weekly shop'));
  await page.getByRole('button', { name: 'Rename list' }).click();
  await expect(page.getByRole('status')).toHaveText('List renamed to Weekly shop');
});
```

Run it: `npx playwright test`

Lesson: Alerts, confirms and prompts

Catch a download and read the file

tests/download.spec.js

```
import { test, expect } from '@playwright/test';
import { readFileSync } from 'node:fs';

test('download the orders report', async ({ page }) => {
  await page.goto('/widgets');
  const downloadPromise = page.waitForEvent('download');
  await page.getByRole('link', { name: 'Download report' }).click();
  const download = await downloadPromise;
```

```
  console.log('File name:', download.suggestedFilename());
  const text = readFileSync(await download.path(), 'utf8');
  console.log(text.trim());
  expect(text.split('\n')[0]).toBe('order,total');
  await download.saveAs('reports/' + download.suggestedFilename());   // keep a copy
});
```

Run it: `npx playwright test`

Lesson: Downloads and uploads

Upload files without touching the disk

tests/upload.spec.js

```
import { test, expect } from '@playwright/test';

test('send feedback with two screenshots', async ({ page }) => {
  await page.goto('/forms');
  await page.getByPlaceholder('Full name').fill('Asha');
  await page.getByLabel('Screenshots').setInputFiles([
    { name: 'cart.png', mimeType: 'image/png', buffer: Buffer.from('fake picture 1') },
    { name: 'error.png', mimeType: 'image/png', buffer: Buffer.from('fake picture 2') },
  ]);
  await page.getByRole('button', { name: 'Send feedback' }).click();
  await expect(page.getByRole('status')).toContainText('files: cart.png, error.png');
});
```

Run it: `npx playwright test`

Lesson: Downloads and uploads

Listen to console messages, page errors and requests

tests/events.spec.js

```
import { test, expect } from '@playwright/test';

test('catch console messages and page errors', async ({ page }) => {
  const messages = [];
  page.on('console', msg => messages.push(`${msg.type()}: ${msg.text()}`));
  page.on('request', request => {
    if (request.url().includes('/api/')) console.log('request →', request.method(), request.url());
  });
  await page.goto('/widgets');

  const warning = page.waitForEvent('console', msg => msg.type() === 'warning');
  await page.getByRole('button', { name: 'Write to the console' }).click();
  await warning;
  console.log(messages);

  const pageError = page.waitForEvent('pageerror');
  await page.getByRole('button', { name: 'Break the page' }).click();
  console.log('Page error:', (await pageError).message);
});
```

Run it: `npx playwright test`

Lesson: Listening to what the page does

Run JavaScript inside the page

tests/evaluate.spec.js

```
import { test, expect } from '@playwright/test';

test('run code inside the page', async ({ page }) => {
  // Runs in the page before any of its own scripts, on every navigation.
  await page.addInitScript(() => { window.testMode = true; });
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);   // wait for the list

  const info = await page.evaluate(() => ({
    title: document.title,
    cards: document.querySelectorAll('li.card').length,
    testMode: window.testMode,
  }));
  console.log(info);
```

```
  const sum = await page.evaluate(([a, b]) => a + b, [20, 199]);   // pass values in
  console.log('sum =', sum);

  const tag = await page.getByPlaceholder('Search products').evaluate(el => el.tagName);
  console.log('tag =', tag);
});
```

Run it: `npx playwright test`

Lesson: Running code inside the page

Click a button inside a shadow DOM component

tests/shadow.spec.js

```
import { test, expect } from '@playwright/test';

test('vote inside a shadow DOM component', async ({ page }) => {
  await page.goto('/shadow');
  const ashaReview = page.locator('review-card').filter({ hasText: 'Asha' });

  // getByRole and CSS look inside open shadow roots by themselves.
  await ashaReview.getByRole('button', { name: 'Helpful' }).click();
  await expect(ashaReview.getByText('found this helpful')).toHaveText('1 person found this helpful');

  console.log('CSS sees the buttons:', await page.locator('review-card button').count());
  console.log('XPath sees the buttons:', await page.locator('//review-card//button').count());
});
```

Run it: `npx playwright test`

Lesson: Shadow DOM and web components

TABLES

Table: dialog · opened by · accept() · dismiss()

Dialog	Opened by	accept()	dismiss()
alert	`alert(msg)`	closes it	closes it
confirm	`confirm(msg)`	the page gets true	the page gets false
prompt	`prompt(msg, default)`	the page gets your text	the page gets null
beforeunload	leaving a page with unsaved changes	leave the page	stay on the page

Lesson: Alerts, confirms and prompts

Table: event · tells you · typical use

Event	Tells you	Typical use
console	a message the page logged	fail on `console.error`, debug a test
pageerror	an error nothing caught	fail the test on broken JavaScript
request and response	every network call	check an API was called, count calls
popup, download, dialog	a new tab, a file, a dialog	handle it in the test
close, crash	the page closed or crashed	clean up, report a crash

Lesson: Listening to what the page does

TIPS AND TRAPS

COMMON MISTAKE Do not wait for the network to go quiet. A chat widget or analytics script can keep it busy forever. Wait for what the user is waiting for: a URL, a heading or a message. Every check with expect already waits.

Lesson: Going to pages and waiting for them

REMEMBER Some pages hide the real file input and open the file window from a pretty button. Then wait for the file chooser event: `page.waitForEvent('filechooser')` in JavaScript, and call `setFiles` on it.

Lesson: Downloads and uploads

IN REAL TEAMS Many companies add one shared fixture that collects page errors and fails any test that had one. It finds JavaScript bugs on pages nobody wrote a check for.

Lesson: Listening to what the page does

COMMON MISTAKE Use evaluate to read and to prepare, not to click. A click done with JavaScript skips the waiting and the visibility checks that make Playwright tests reliable, and it can pass on a button a real user could never press.

Lesson: Running code inside the page

STEP 3 Finding and using elements

3.1 Locators: finding elements

Why a locator is searched again every time, a live locator lab, which locator to try first, getByRole with names, levels and states, labels, placeholders, text, titles and test ids, custom test id attributes, CSS and XPath and why they break, filter, chaining, nth, and, or, strict mode violations, and a recorded walk through a sortable, paged table.

KEY IDEAS

- **Role and name** What the user sees. The first choice.
- **Forms** Boxes by their label. Placeholder when there is none.
- **Narrow down** Card first, then button. hasText, has, hasNot.
- **One element** Actions need exactly one. The error lists them all.
- **Remember** A locator is searched again every time you use it, and an action needs exactly one match.

CODE

A locator is found again every time you use it

tests/lazy.spec.js

```
import { test, expect } from '@playwright/test';

test('a locator is a recipe, not an element', async ({ page }) => {
  await page.goto('/products');
  const cards = page.getByRole('listitem');          // nothing is searched yet
  await expect(cards).toHaveCount(5);
  console.log('before search:', await cards.count());

  await page.getByPlaceholder('Search products').fill('o');
  console.log('after "o":', await cards.count());    // the same locator, found again

  await page.getByPlaceholder('Search products').fill('bottle');
  await expect(cards).toHaveCount(1);
  console.log('after "bottle":', await cards.count());
  console.log(cards.describe('product cards').toString());
});
```

Run it: `npx playwright test`

Lesson: What a locator is

getByRole with names, levels and states

tests/roles.spec.js

```
import { test, expect } from '@playwright/test';

test('find things the way a user sees them', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);   // waits for the list
```

```
  console.log('headings:', await page.getByRole('heading').allTextContents());
  console.log('level 1 heading:', await page.getByRole('heading', { level: 1 }).textContent());

  // The name is matched ignoring case, and part of the name is enough.
  console.log('"add to cart":', await page.getByRole('button', { name: 'add to cart' }).count());
  // exact: true wants the whole name with the same case.
  console.log('"add to cart" exact:', await page.getByRole('button', { name: 'add to cart', exact: true
}).count());
  console.log('disabled buttons:', await page.getByRole('button', { disabled: true }).count());

  await page.getByRole('searchbox', { name: 'Search' }).fill('lamp');
  await page.getByRole('combobox', { name: 'Sort by' }).selectOption('high');
  await expect(page.getByRole('listitem')).toHaveCount(1);
  console.log('left after search:', await page.getByRole('listitem').getByRole('heading').textContent());
});
```

Run it: `npx playwright test`

Lesson: getByRole: the first choice

Labels, placeholders, text, titles and test ids

tests/getby.spec.js

```
import { test, expect } from '@playwright/test';

test('labels, placeholders, text, titles and test ids', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByText('Remember me').click();          // clicking a label ticks its box
  await expect(page.getByLabel('Remember me')).toBeChecked();
  await page.getByRole('button', { name: 'Sign in' }).click();

  await expect(page.getByText('Hello, Asha')).toBeVisible();
  await expect(page.getByTestId('cart-count')).toHaveText('0');
  await expect(page.getByPlaceholder('Search products')).toBeEditable();
  await expect(page.getByTestId(/^product-/)).toHaveCount(5);   // all product cards

  await page.goto('/frames');
  console.log('iframes by title:', await page.getByTitle('Payment form').count());
});
```

Run it: `npx playwright test`

Lesson: Labels, placeholders, text, alt text, titles and test ids

CSS and XPath

tests/css.spec.js

```
import { test, expect } from '@playwright/test';

test('CSS and XPath when nothing better fits', async ({ page }) => {
  await page.goto('/products');
  await expect(page.locator('li.card')).toHaveCount(5);

  console.log('CSS prices:', await page.locator('li.card .price').allTextContents());
  console.log('CSS attribute:', await page.locator('[data-testid="product-3"] h3').textContent());
  console.log('CSS :has-text():', await page.locator('li.card:has-text("mug") h3').textContent());
  console.log('XPath:', await page.locator('//li[h3="Desk lamp"]//span[@class="price"]').textContent());
  console.log('id:', await page.locator('#sort').inputValue());
});
```

Run it: `npx playwright test`

Lesson: CSS and XPath

filter, first, nth, last, and, or

tests/filter.spec.js

```
import { test, expect } from '@playwright/test';

test('narrow down with filter, chaining, and and or', async ({ page }) => {
  await page.goto('/products');
  const cards = page.getByRole('listitem');
  await expect(cards).toHaveCount(5);
```

```
  const mug = cards.filter({ hasText: 'Coffee mug' });
  console.log('mug price:', await mug.locator('.price').textContent());

  const kitchen = cards.filter({ hasText: 'Kitchen' });
  console.log('kitchen:', await kitchen.getByRole('heading').allTextContents());

  const soldOut = cards.filter({ has: page.getByRole('button', { name: 'Out of stock' }) });
  console.log('sold out:', await soldOut.getByRole('heading').textContent());

  const inStock = cards.filter({ hasNot: page.getByRole('button', { disabled: true }) });
  console.log('in stock:', await inStock.count());

  console.log('first:', await cards.first().getByRole('heading').textContent());
  console.log('nth(2):', await cards.nth(2).getByRole('heading').textContent());
  console.log('last:', await cards.last().getByRole('heading').textContent());

  const anyButton = page.getByRole('button', { name: 'Add to cart' }).or(page.getByRole('button', { name:
'Out of stock' }));
  console.log('or:', await anyButton.count());
  const disabledButton = page.getByRole('button').and(page.locator('[disabled]'));
  console.log('and:', await disabledButton.textContent());
});
```

Run it: `npx playwright test`

Lesson: Filtering and chaining

Strict mode: one action, one element

tests/strict.spec.js

```
import { test, expect } from '@playwright/test';

test('strict mode refuses to guess', async ({ page }) => {
  await page.goto('/products');
  const add = page.getByRole('button', { name: 'Add to cart' });
  await expect(add).toHaveCount(4);              // counting many is fine

  const error = await add.click().catch(e => e);  // acting on many is not
  console.log(error.message.split('\n').slice(0, 4).join('\n'));
});
```

Run it: `npx playwright test`

Lesson: Strict mode

Working on one row of the orders table

`await page.goto('http://localhost:4173/orders');`	Open the orders page. It shows 5 of 7 orders per page.
`await page.getByRole('button', { name: 'Total' }).click();`	Click the Total column header to sort by total, smallest first.
`await page.getByRole('row').filter({ hasText: 'John' }).getByRole('button', { name: 'Delete' }).click();`	Find the row that mentions John, then the Delete button inside that row only.
`await expect(page.getByRole('status')).toHaveText('Order #1004 deleted');`	The status message confirms which order went away.
`await page.getByRole('button', { name: 'Next' }).click();`	Go to the next page of the table.
`await expect(page.getByText('Page 2 of 2')).toBeVisible();`	Check the page counter by its text.

Lesson: Lists and tables

Read every row of a table, then act on one row

tests/table.spec.js

```
import { test, expect } from '@playwright/test';

test('read a table and act on one row', async ({ page }) => {
  await page.goto('/orders');
  const rows = page.getByRole('row').filter({ has: page.getByRole('cell') });   // skip the header row
  await expect(rows).toHaveCount(5);

  for (const row of await rows.all()) {
```

```
    console.log(await row.getByRole('cell').allTextContents());
  }

  const meera = rows.filter({ hasText: 'Meera' });
  await meera.getByRole('button', { name: 'Delete' }).click();
  await expect(page.getByRole('status')).toHaveText('Order #1003 deleted');
  await expect(rows.filter({ hasText: 'Meera' })).toHaveCount(0);
});
```

Run it: `npx playwright test`

Lesson: Lists and tables

TABLES

Table: html · role · example locator

HTML	Role	Example locator
<button>, <input type=submit>	button	getByRole('button', { name: 'Sign in' })
<a href>	link	getByRole('link', { name: 'Orders' })
<h1> to <h6>	heading	getByRole('heading', { level: 1 })
<input type=text>, <textarea>	textbox	getByRole('textbox', { name: 'Email' })
<input type=search>	searchbox	getByRole('searchbox')
<select>	combobox	getByRole('combobox', { name: 'Sort by' })
<input type=checkbox> and radio	checkbox, radio	getByRole('checkbox', { checked: true })
, <tr>, <td>	listitem, row, cell	getByRole('row').filter(...)

Lesson: getByRole: the first choice

Table: locator · finds · good for

Locator	Finds	Good for
getByLabel	a form box by its <label> or aria-label	every form
getByPlaceholder	a box by its hint text	boxes with no label
getByText	the smallest element that contains the text	messages, paragraphs, labels
getByAltText	an image by its alt text	logos, product pictures
getByTitle	an element by its title attribute	icons, iframes
getByTestId	an element by data-testid	text that changes or is translated

Lesson: Labels, placeholders, text, alt text, titles and test ids

Table: you want · better choice · fragile choice

You want	Better choice	Fragile choice
the Sign in button	getByRole('button', { name: 'Sign in' })	locator('#login > button.btn-primary')
the email box	getByLabel('Email')	locator('//form/div[1]/input')
the price of the mug	getByRole('listitem').filter({ hasText: 'Coffee mug' }).getByText(/₹/)	locator('li:nth-child(1) span:nth-child(3)')

Lesson: CSS and XPath

Table: method · keeps

Method	Keeps
filter({ hasText: 'x' })	elements that contain the text somewhere inside
hasNotText: 'x'	elements that do not contain the text
has: locator	elements that contain an element the locator finds
hasNot: locator	elements that do not contain it
visible: true	only visible elements
first(), last(), nth(i)	one element by position, nth counts from 0
a.and(b), a.or(b)	elements both match, or either matches

Lesson: Filtering and chaining

TIPS AND TRAPS

IN REAL TEAMS If getByRole cannot find a button that you can see, the page is probably missing a role or a name, and a screen reader user cannot use it either. That is a real accessibility bug worth reporting.

Lesson: getByRole: the first choice

IN REAL TEAMS Many companies use another attribute for test ids, such as data-qa or data-test. Tell Playwright once: `use: { testIdAttribute: 'data-qa' }` in playwright.config. After that, getByTestId reads the new attribute.

Lesson: Labels, placeholders, text, alt text, titles and test ids

COMMON MISTAKE Long XPaths copied from the browser tools, such as /html/body/div[2]/div/form/input, break as soon as one box is added to the page. Recorded tests from older tools are full of them, and they are the first thing to replace when a team moves to Playwright.

Lesson: CSS and XPath

COMMON MISTAKE first(), last() and nth() depend on the order of the page. When the order changes, they quietly pick another element. Prefer a filter by text when you can.

Lesson: Filtering and chaining

3.2 Actions and auto-waiting

An interactive auto-waiting lab, a real click on a covered, disabled and moving button, the actionability table from the docs, the call log of a timeout, force and trial, click options and hover, fill versus pressSequentially, a recorded walk through a feedback form, every form control, drag and drop by dragTo and by hand, and waits that replace sleeps.

KEY IDEAS

- **Auto-waiting** Checks before every action. No sleeps needed.
- **Timeouts** The error says which check failed. Read it before using force.
- **Typing** fill for most boxes. pressSequentially for key by key.
- **Forms** check, selectOption, dragTo. Found by their labels.
- **Remember** Never sleep. Every action waits until the element is ready, and when you need more, wait for the exact thing you need.

CODE

Click a button that is covered, disabled and moving

tests/auto-wait.spec.js

```
import { test, expect } from '@playwright/test';

test('actions wait until the element is ready', async ({ page }) => {
  await page.goto('/widgets');
  // A loading cover hides the page for 0.8 s, and Pay now is disabled for 1 s.
```

```
  await page.getByRole('button', { name: 'Pay now' }).click();
  await expect(page.getByText('Payment done')).toBeVisible();

  // The offer appears 1.2 s after the click. The check waits for it.
  await page.getByRole('button', { name: 'Load offers' }).click();
  await expect(page.getByTestId('offer')).toHaveText('Buy 2 mugs, get 1 free');

  // This button slides in when the page opens. click waits until it stops moving.
  await page.getByRole('button', { name: 'Catch me' }).click();
  await expect(page.getByText('Caught it')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: Auto-waiting: no more sleeps

A click that can never happen, and its call log

tests/call-log.spec.js

```
import { test, expect } from '@playwright/test';

test('when it never gets ready, the error says why', async ({ page }) => {
  await page.goto('/products');
  const soldOut = page.getByRole('button', { name: 'Out of stock' });

  const error = await soldOut.click({ timeout: 2000 }).catch(e => e);
  console.log(error.name);
  console.log(error.message.split('\n').slice(0, 7).join('\n'));
});
```

Run it: `npx playwright test`

Lesson: When waiting is not enough

Hover and focus

tests/hover.spec.js

```
import { test, expect } from '@playwright/test';

test('hover and focus show the tooltip', async ({ page }) => {
  await page.goto('/forms');
  const help = page.getByText('❓ Help');
  const tooltip = page.getByRole('tooltip');
  await expect(tooltip).toBeHidden();

  await help.hover();                          // the mouse rests on it
  await expect(tooltip).toHaveText('We reply within two working days');

  await page.mouse.move(0, 0);                 // move the mouse away
  await expect(tooltip).toBeHidden();

  await help.focus();                          // keyboard users reach it with Tab
  await expect(tooltip).toBeVisible();
  console.log('tooltip:', await tooltip.textContent());
});
```

Run it: `npx playwright test`

Lesson: Clicking, hovering and focusing

fill, pressSequentially and keyboard shortcuts

tests/keys.spec.js

```
import { test, expect } from '@playwright/test';

test('fill, type key by key, and press shortcuts', async ({ page }) => {
  await page.goto('/products');
  const search = page.getByPlaceholder('Search products');
  await expect(page.getByRole('listitem')).toHaveCount(5);

  await search.fill('desk');                                  // sets the whole value at once
  console.log('after fill:', await search.inputValue(), '|', await page.getByRole('listitem').count(),
'card');

  await search.clear();
  await search.pressSequentially('mug', { delay: 50 });    // one key at a time, like a person
```

```
  console.log('after typing:', await search.inputValue());

  await search.press('Control+A');
  await search.press('Backspace');
  console.log('after select all and delete:', (await search.inputValue()).length, 'characters');

  await page.goto('/forms');
  await page.keyboard.press('Control+K');                // a shortcut for the whole page
  await expect(page.getByText('Search opened')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: Typing and keys

Filling the feedback form

`await page.goto('http://localhost:4173/forms');`	Open the feedback form.
`await page.getByLabel('Your name').fill('Asha');`	Type the name into the box labelled Your name.
`await page.getByLabel('Topic').selectOption('delivery');`	Choose Delivery in the Topic drop down.
`await page.getByLabel('Happy').check();`	Pick the Happy radio button.
`await page.getByLabel('Message').fill('The parcel came a day early.');`	Write a short message.
`await page.getByLabel('Send me the newsletter').check();`	Tick the newsletter box.
`await page.getByRole('button', { name: 'Send feedback' }).click();`	Send the form. Playwright scrolls the button into view by itself.
`await expect(page.getByRole('status')).toContainText('Topic: delivery, mood: happy');`	The thank you line shows every value the page received.

Lesson: Checkboxes, radio buttons, drop downs and dates

Text boxes, drop downs, radio buttons, checkboxes, dates and sliders

tests/form.spec.js

```
import { test, expect } from '@playwright/test';

test('fill every kind of form control', async ({ page }) => {
  await page.goto('/forms');
  await page.getByLabel('Your name').fill('Asha');
  await page.getByLabel('Topic').selectOption('delivery');            // by value
  await page.getByLabel('Topic').selectOption({ label: 'Payment' });  // or by the text you see
  await page.getByLabel('Happy').check();                             // a radio button
  await page.getByLabel('Order date').fill('2026-10-01');             // dates use yyyy-mm-dd
  await page.getByLabel('Message').fill('The parcel came a day early.');
  await page.getByLabel('Send me the newsletter').setChecked(true);
  await page.getByLabel('Volume').fill('8');                          // a slider

  console.log('topic:', await page.getByLabel('Topic').inputValue());
  console.log('happy checked:', await page.getByLabel('Happy').isChecked());
  console.log('date:', await page.getByLabel('Order date').inputValue());
  console.log('volume:', await page.getByLabel('Volume').inputValue());

  await page.getByRole('button', { name: 'Send feedback' }).click();
  await expect(page.getByRole('status')).toHaveText(
    'Thanks Asha! Topic: payment, mood: happy, files: no files, newsletter: yes');
});
```

Run it: `npx playwright test`

Lesson: Checkboxes, radio buttons, drop downs and dates

Drag and drop with dragTo, and by hand with the mouse

tests/drag.spec.js

```
import { test, expect } from '@playwright/test';

test('drag tasks to the Done lane', async ({ page }) => {
  await page.goto('/forms');
  const todo = page.getByRole('list', { name: 'To do' });
  const done = page.getByRole('list', { name: 'Done' });
```

```
  await todo.getByText('Pack the order').dragTo(done);
  await expect(done.getByRole('listitem')).toHaveText(['Pack the order']);

  // The same by hand with the mouse, for widgets that need finer control.
  await todo.getByText('Print the label').hover();
  await page.mouse.down();
  await done.hover();
  await done.hover();            // a second move, so the page gets its dragover event
  await page.mouse.up();

  await expect(done.getByRole('listitem')).toHaveCount(2);
  console.log('Done lane:', await done.getByRole('listitem').allTextContents());
  console.log('To do lane:', await todo.getByRole('listitem').count(), 'tasks');
});
```

Run it: `npx playwright test`

Lesson: Drag and drop and the mouse

Wait for a response, an element or a condition

tests/wait.spec.js

```
import { test, expect } from '@playwright/test';

test('wait for what you need, never for a fixed time', async ({ page }) => {
  // Start waiting for the API answer before the page asks for it.
  const answer = page.waitForResponse(r => r.url().includes('/api/products'));
  await page.goto('/products');
  console.log('API answered with status', (await answer).status());

  await page.getByText('Loading products…').waitFor({ state: 'detached' });
  console.log('cards now:', await page.getByRole('listitem').count());

  await page.goto('/widgets');
  await page.waitForFunction(() => !document.querySelector('#cover'));
  console.log('loading cover left:', await page.locator('#cover').count());
  await expect(page.getByRole('button', { name: 'Pay now' })).toBeEnabled();
});
```

Run it: `npx playwright test`

Lesson: Waiting without sleeping

TABLES

Table: action · visible · stable · receives events

Action	Visible	Stable	Receives events	Enabled	Editable
click, dblclick, check, uncheck, setChecked, tap	✓	✓	✓	✓	
hover, dragTo	✓	✓	✓		
screenshot	✓	✓			
fill, clear	✓			✓	✓
selectOption	✓			✓	
selectText	✓				
scrollIntoViewIfNeeded		✓			
focus, blur, press, pressSequentially, setInputFiles, dispatchEvent					

Lesson: Auto-waiting: no more sleeps

Table: option · what it does · when to use it

Option	What it does	When to use it
force: true	skips the checks that are not essential, such as receives events	a custom widget that hides its real input under a styled label
trial: true	runs every check but does not click	to wait until something could be clicked, without clicking it
timeout: 5000	a different timeout for this one action	one action that is known to be slow

Lesson: When waiting is not enough

Table: you want · code

You want	Code
a double click	dblclick()
a right click	click({ button: 'right' })
Shift and click	click({ modifiers: ['Shift'] })
Control on Windows and Linux, Command on macOS	modifiers: ['ControlOrMeta']
a click at the top left corner	position: { x: 0, y: 0 }
the mouse resting on it	hover()
a tap on a touch screen	tap() with hasTouch: true in use

Lesson: Clicking, hovering and focusing

Table: method · sends · use for

Method	Sends	Use for
fill(text)	the whole value at once	almost every text box
pressSequentially(text)	one key event per character	auto complete, masks that format as you type
press(key)	one key or a combination	Enter, Tab, Escape, ArrowDown, Control+A
keyboard.press(key)	a key to whatever has focus	page shortcuts such as Control+K
clear()	an empty value	emptying a box

Lesson: Typing and keys

Table: control · method · read it back

Control	Method	Read it back
text box, text area	fill	inputValue
checkbox, radio button	check, uncheck, setChecked	isChecked or toBeChecked
drop down (select)	selectOption by value, label or index, or a list for multi select	inputValue or toHaveValue
date, time, color, range	fill with the standard format, such as 2026-10-01	inputValue
file input	setInputFiles	the files list

Lesson: Checkboxes, radio buttons, drop downs and dates

Table: wait for · method

Wait for	Method
an element to appear, hide or go away	locator.waitFor({ state })
a new URL	page.waitForURL
a network response	page.waitForResponse(...)
anything the page can compute	page.waitForFunction
a popup, download or dialog	page.waitForEvent('popup')

Lesson: Waiting without sleeping

TIPS AND TRAPS

COMMON MISTAKE Reaching for force is often a sign the test found the wrong element, or that a real user could not click it either. Read the call log first.

Lesson: When waiting is not enough

NEW IN PLAYWRIGHT Old tutorials use type(). It was renamed pressSequentially in Playwright 1.38 and the old name is deprecated. If you see it in a code base, it does the same thing.

Lesson: Typing and keys

IN REAL TEAMS Many teams add a lint rule or a code review check that rejects waitForTimeout. A suite with 200 tests and a 2 second sleep in each spends almost 7 minutes doing nothing on every run.

Lesson: Waiting without sleeping

3.3 Assertions that wait for the page

How a web-first assertion retries, a plain read against a retrying check, fifteen checks on one page, text and list checks for sorting, negative checks, a real failure report, soft assertions in all four languages, plain value checks on API data, expect.poll and toPass, and a recorded walk through a search.

KEY IDEAS

- **Web-first** Retries until true. 5 seconds by default.
- **Text** Exact or contains. Lists check order too.
- **Negative** Waits for things to go. toHaveCount(0) for none.
- **Soft** Keep going. Report every problem.
- **Remember** Check the page with assertions that retry, and never read a value first and check it later.

CODE

A web-first check against a plain read

tests/retry.spec.js

```
import { test, expect } from '@playwright/test';

test('a web-first check waits, a plain read does not', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Load offers' }).click();
  const offer = page.getByTestId('offer');      // shows up 1.2 s after the click

  console.log('isVisible() right now:', await offer.isVisible());   // a snapshot, no waiting
  await expect(offer).toBeVisible();                                 // retries until it shows
  console.log('isVisible() after the check:', await offer.isVisible());
});
```

Run it: `npx playwright test`

Lesson: Web-first assertions retry

Fifteen checks on the sign in page

tests/many-checks.spec.js

```
import { test, expect } from '@playwright/test';

test('one check for each thing a user can notice', async ({ page }) => {
  await page.goto('/login');
  await expect(page).toHaveTitle('Sign in · Brainy Shop');
  await expect(page).toHaveURL(/\/login$/);

  const email = page.getByLabel('Email');
  await expect(email).toBeEmpty();
  await expect(email).toBeEditable();
  await expect(email).toHaveAttribute('type', 'email');
  await expect(email).toHaveRole('textbox');
  await email.fill('asha@shop.example');
  await expect(email).toHaveValue('asha@shop.example');
  await expect(email).toBeFocused();

  await expect(page.getByRole('checkbox', { name: 'Remember me' })).not.toBeChecked();
  await expect(page.getByRole('alert')).toBeHidden();
  const signIn = page.getByRole('button', { name: 'Sign in' });
  await expect(signIn).toBeEnabled();
  await expect(signIn).toHaveCSS('background-color', 'rgb(22, 163, 74)');
  await expect(signIn).toBeInViewport();
  await expect(page.getByRole('form')).toHaveAccessibleName('Sign in');
  console.log('all 15 checks passed');
});
```

Run it: `npx playwright test`

Lesson: The assertions you will use every day

toHaveText and toContainText for one element and for lists

tests/text.spec.js

```
import { test, expect } from '@playwright/test';

test('text checks for one element and for a list', async ({ page }) => {
  await page.goto('/products');
  const names = page.getByRole('listitem').getByRole('heading');

  // A list: every element, in this order.
  await expect(names).toHaveText(['Coffee mug', 'Desk lamp', 'Gel pen', 'Notebook', 'Water bottle']);
  // A list: some elements contain these texts, in this order.
  await expect(names).toContainText(['mug', 'pen']);

  await expect(page.getByRole('heading', { level: 1 })).toHaveText('products', { ignoreCase: true });
  await expect(page.getByTestId('product-3')).toContainText('₹1,299');
  await expect(page.getByTestId('product-3')).toHaveText(/Desk lamp.*Home/);

  await page.getByRole('combobox', { name: 'Sort by' }).selectOption('low');
  await expect(names).toHaveText(['Gel pen', 'Notebook', 'Coffee mug', 'Water bottle', 'Desk lamp']);
  console.log('sorted:', await names.allTextContents());
});
```

Run it: `npx playwright test`

Lesson: Checking text and lists

Checking a search, one assertion at a time

`await page.goto('http://localhost:4173/products');`	Open the products page.
`await expect(page.getByRole('listitem')).toHaveCount(5);`	The check waits until the slow product list has loaded.
`await page.getByPlaceholder('Search products').fill('bottle');`	Search for bottle.
`await expect(page.getByRole('listitem')).toContainText('Water bottle');`	One card is left, and it is the right one.
`await expect(page.getByPlaceholder('Search products')).toHaveValue('bottle');`	The search box keeps what was typed.

`await page.getByPlaceholder('Search products').fill('phone');`	Search for something the shop does not sell.
`await expect(page.getByRole('button', { name: 'Add to cart' })).toHaveCount(0);`	No Add to cart buttons are left. A negative check also waits.

Lesson: Checking that something is not there

Checks that something is gone

tests/negative.spec.js

```
import { test, expect } from '@playwright/test';

test('negative checks wait too', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByText('Loading products…')).not.toBeVisible();   // waits until it goes away
  await expect(page.getByText('No products match your search')).toHaveCount(0);

  await page.getByPlaceholder('Search products').fill('phone');
  await expect(page.getByRole('listitem')).toHaveText('No products match your search');
  await expect(page.getByRole('button', { name: 'Add to cart' })).toHaveCount(0);
  await expect(page.getByRole('button', { name: 'Add to cart' })).not.toBeAttached();
  console.log('nothing to buy when searching for phone');
});
```

Run it: `npx playwright test`

Lesson: Checking that something is not there

What a failing check looks like

tests/failing.spec.js

```
import { test, expect } from '@playwright/test';

test('a failing check explains itself', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
  await expect(page.getByTestId('product-2'), 'the mug should cost 149 in the sale')
    .toContainText('₹149', { timeout: 2000 });
});
```

Run it: `npx playwright test`

Lesson: Reading a failure

Soft checks: keep going, then fail with every problem

tests/soft.spec.js

```
import { test, expect } from '@playwright/test';

test('soft checks report every problem at once', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);

  await expect.soft(page.getByTestId('product-1')).toContainText('₹25', { timeout: 1000 });          //
wrong on purpose
  await expect.soft(page.getByTestId('product-4')).toContainText('Add to cart', { timeout: 1000 });  //
wrong on purpose
  await expect.soft(page.getByTestId('product-5')).toContainText('₹349');                            // right
  console.log('the test kept going after', test.info().errors.length, 'failed checks');
});
```

Run it: `npx playwright test`

Lesson: Soft assertions

Plain checks on data you already have

tests/values.spec.js

```
import { test, expect } from '@playwright/test';

test('plain value checks for data you already have', async ({ page }) => {
  const response = await page.request.get('/api/products?delay=0');
  await expect(response).toBeOK();                  // a web-first check for API answers
  const products = await response.json();
```

```
  expect(products).toHaveLength(5);                 // plain checks: no waiting, no await
  expect(products.map(p => p.name)).toContain('Desk lamp');
  expect(products[0]).toEqual({ id: 1, name: 'Gel pen', price: 20, stock: 120, category: 'Stationery' });
  expect(products[2].price).toBeGreaterThan(1000);
  console.log('cheapest:', Math.min(...products.map(p => p.price)));
});
```

Run it: `npx playwright test`

Lesson: Plain values, polling and blocks

expect.poll, toPass and expect.configure

tests/poll.spec.js

```
import { test, expect } from '@playwright/test';

test('expect.poll and toPass retry your own code', async ({ page }) => {
  await page.goto('/products');
  // Poll any value until it matches: here a count read with a plain call.
  await expect.poll(() => page.locator('li.card').count(), { timeout: 5000 }).toBe(5);

  // Retry a whole block until every check inside it passes.
  let tries = 0;
  await expect(async () => {
    tries++;
    const response = await page.request.get('/api/products?delay=0');
    expect(response.status()).toBe(200);
    expect(tries).toBeGreaterThanOrEqual(3);     // the first two tries fail on purpose
  }).toPass({ intervals: [100, 200, 500] });
  console.log('the block passed on try', tries);

  const slowExpect = expect.configure({ timeout: 10_000 });   // a copy of expect with other defaults
  await slowExpect(page.getByRole('heading', { level: 1 })).toHaveText('Products');
});
```

Run it: `npx playwright test`

Lesson: Plain values, polling and blocks

TABLES

Table: checks · javascript and typescript

Checks	JavaScript and TypeScript
visible or hidden	toBeVisible, toBeHidden
text	toHaveText, toContainText
how many	toHaveCount
box value	toHaveValue, toHaveValues, toBeEmpty
state	toBeChecked, toBeEnabled, toBeDisabled, toBeEditable, toBeFocused
attributes and style	toHaveAttribute, toHaveClass, toContainClass, toHaveCSS, toHaveId
accessibility	toHaveRole, toHaveAccessibleName, toHaveAccessibleDescription
the page	toHaveTitle, toHaveURL
an API answer	toBeOK
pictures and structure	toHaveScreenshot, toMatchAriaSnapshot

Lesson: The assertions you will use every day

Table: assertion · with a string · with a list of strings

Assertion	With a string	With a list of strings
toHaveText	the whole text must match	exactly these elements, in this order
toContainText	the text must contain it	some elements, in this order, containing these parts

Lesson: Checking text and lists

TIPS AND TRAPS

COMMON MISTAKE A rule that catches many flaky tests: never put await inside expect. Write `await expect(locator).toBeVisible()`, not `expect(await locator.isVisible()).toBe(true)`.

Lesson: Web-first assertions retry

REMEMBER Change the timeout of one check with its timeout option, or of all checks: `expect: { timeout: 10_000 }` in playwright.config. Raise it for a slow test environment, not to hide a slow page.

Lesson: Reading a failure

IN REAL TEAMS Soft checks are handy for a page with many independent fields, such as an invoice or a profile. Do not use them for steps the rest of the test depends on, such as signing in.

Lesson: Soft assertions

STEP 4 The test runner

4.1 The test runner: organise, tag, parameterise and configure

Which runner each language uses, what happens in a test run, groups, tags, skip and expected failures in all four languages, running only smoke tests, parameterised search tests, and one test run as a desktop and as a phone from configuration.

KEY IDEAS

- **Runner** Finds and runs tests. One per language.
- **Tags** Run a slice. Smoke now, regression tonight.
- **Data** One test, many rows. Each row is its own result.
- **Config** Settings in one place. Projects for each device.
- **Remember** Tests say what to check. The runner and its configuration decide where, how and with what settings they run.

CODE

Group, tag, skip and mark known failures

tests/products-page.spec.js

```
import { test, expect } from '@playwright/test';

test.describe('products page', () => {
  test('shows five products', { tag: '@smoke' }, async ({ page }) => {
    await page.goto('/products');
    await expect(page.getByRole('listitem')).toHaveCount(5);
  });

  test('sorts by price', { tag: '@regression' }, async ({ page }) => {
    await page.goto('/products');
    await page.getByLabel('Sort by').selectOption('low');
    await expect(page.getByRole('listitem').first()).toContainText('Gel pen');
  });

  test.skip('adds a product to the wish list', async ({ page }) => {
    // the wish list is not built yet
  });

  test('sells phones', {
    tag: '@regression',
    annotation: { type: 'issue', description: 'SHOP-42: phones arrive next month' },
  }, async ({ page }) => {
    test.fail();   // known to fail today, so a failure is the expected result
    await page.goto('/products');
    await page.getByPlaceholder('Search products').fill('phone');
    await expect(page.getByRole('button', { name: 'Add to cart' })).toHaveCount(1, { timeout: 1000 });
```

```
  });
});
```

Run it: `npx playwright test`

Lesson: Grouping, tags and annotations

Run only the tests with one tag

tests/products-page.spec.js

```
import { test, expect } from '@playwright/test';

test.describe('products page', () => {
  test('shows five products', { tag: '@smoke' }, async ({ page }) => {
    await page.goto('/products');
    await expect(page.getByRole('listitem')).toHaveCount(5);
  });

  test('sorts by price', { tag: '@regression' }, async ({ page }) => {
    await page.goto('/products');
    await page.getByLabel('Sort by').selectOption('low');
    await expect(page.getByRole('listitem').first()).toContainText('Gel pen');
  });

  test.skip('adds a product to the wish list', async ({ page }) => {
    // the wish list is not built yet
  });

  test('sells phones', {
    tag: '@regression',
    annotation: { type: 'issue', description: 'SHOP-42: phones arrive next month' },
  }, async ({ page }) => {
    test.fail();   // known to fail today, so a failure is the expected result
    await page.goto('/products');
    await page.getByPlaceholder('Search products').fill('phone');
    await expect(page.getByRole('button', { name: 'Add to cart' })).toHaveCount(1, { timeout: 1000 });
  });
});
```

Run it: `npx playwright test --grep @smoke`

Lesson: Running a slice of the suite

One test, many rows of data

tests/search.spec.js

```
import { test, expect } from '@playwright/test';

const searches = [
  { term: 'mug', count: 1, first: 'Coffee mug' },
  { term: 'o', count: 3, first: 'Coffee mug' },
  { term: 'pen', count: 1, first: 'Gel pen' },
  { term: 'BOTTLE', count: 1, first: 'Water bottle' },
];

for (const { term, count, first } of searches) {
  test(`search for "${term}" finds ${count}`, async ({ page }) => {
    await page.goto('/products');
    await page.getByPlaceholder('Search products').fill(term);
    const cards = page.getByRole('listitem');
    await expect(cards).toHaveCount(count);
    await expect(cards.first()).toContainText(first);
  });
}
```

Run it: `npx playwright test`

Lesson: Parameterised tests

Configuration that every test receives

playwright.config.js

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
```

```
  timeout: 30_000,                    // one test may take 30 s
  expect: { timeout: 5_000 },         // one assertion may keep trying for 5 s
  fullyParallel: true,
  forbidOnly: !!process.env.CI,       // a forgotten test.only fails the CI run
  retries: process.env.CI ? 2 : 0,
  workers: 2,
  reporter: 'list',
  use: {
    baseURL: 'http://localhost:4173',
    locale: 'en-IN',
    timezoneId: 'Asia/Kolkata',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'desktop', use: { ...devices['Desktop Chrome'] } },
    { name: 'phone', use: { ...devices['Pixel 7'] } },
  ],
});
```

tests/config.spec.js

```
import { test } from '@playwright/test';

test('the config reaches every test', async ({ page }, testInfo) => {
  await page.goto('/about-you');
  console.log(testInfo.project.name.padEnd(8),
    '| viewport', page.viewportSize()?.width,
    '| language', await page.locator('#lang').textContent(),
    '| time zone', await page.locator('#tz').textContent());
});
```

Run it: `npx playwright test`

Lesson: Configuration in one place

TABLES

Table: language · test runner · how playwright plugs in

Language	Test runner	How Playwright plugs in
JavaScript, TypeScript	Playwright Test (@playwright/test)	built in: fixtures, config, projects, retries, sharding, reporters, UI mode

Lesson: One library, a runner per language

Table: you want · javascript and typescript

You want	JavaScript and TypeScript
a group	test.describe()
a tag	{ tag: '@smoke' }
skip	test.skip()
skip on a condition	test.skip(condition, reason)
known to fail	test.fail()
broken, fix later	test.fixme()
give it more time	test.slow()
run only this one	test.only()

Lesson: Grouping, tags and annotations

Table: setting · playwright test

Setting	Playwright Test
base URL	use: { baseURL }
browser	projects with devices['Desktop Firefox']
headed	--headed or use: { headless: false }
device	use: { ...devices['Pixel 7'] }
locale and other context options	use: { locale }
test timeout	timeout
screenshots, video, trace	use: { screenshot, video, trace }

Lesson: Configuration in one place

TIPS AND TRAPS

REMEMBER Playwright Test was built by the Playwright team, so it has the most features out of the box. pytest, JUnit and TestNG need a plugin or a little code for the same things, which the next chapters show side by side.

Lesson: One library, a runner per language

COMMON MISTAKE A test.only left in the code silently skips every other test. Set forbidOnly in the CI config so such a run fails instead.

Lesson: Grouping, tags and annotations

4.2 Fixtures, hooks and set up once

The order hooks run in, the built in fixtures of each language, the life of a fixture, a signed in page fixture with a clean up, an automatic fixture that fails tests on JavaScript errors, and set up that checks the shop and creates data once per run.

KEY IDEAS

- **Hooks** Around every test. Simple and visible.
- **Fixtures** Only when asked for. Set up, use, tear down.
- **Automatic** Every test gets it. Guards and recordings.
- **Once per run** Health check, data. Before any test.
- **Remember** Put set up where it belongs: in the test if one test needs it, in a fixture if several do, and in a setup step if the whole run does.

CODE

Before and after hooks, and the order they run in

tests/hooks.spec.js

```
import { test, expect } from '@playwright/test';

test.beforeAll(async () => {
  console.log('beforeAll: once for this file');
});

test.beforeEach(async ({ page }, testInfo) => {
  console.log(`beforeEach: open the products page for "${testInfo.title}"`);
  await page.goto('/products');
});

test.afterEach(async ({}, testInfo) => {
  console.log(`afterEach: "${testInfo.title}" ${testInfo.status}`);
});

test.afterAll(async () => {
  console.log('afterAll: once at the end');
```

```
});

test('shows five products', async ({ page }) => {
  await expect(page.getByRole('listitem')).toHaveCount(5);
});

test('finds the lamp', async ({ page }) => {
  await page.getByPlaceholder('Search products').fill('lamp');
  await expect(page.getByRole('listitem')).toHaveCount(1);
});
```

Run it: `npx playwright test`

Lesson: Before and after hooks

A signed in page for every test that asks for it

tests/fixtures.js

```
import { test as base, expect } from '@playwright/test';

export const test = base.extend({
  signedInPage: async ({ page }, use) => {
    // Set up: sign in as Asha.
    await page.goto('/login');
    await page.getByLabel('Email').fill('asha@shop.example');
    await page.getByLabel('Password').fill('s3cret');
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.getByText('Hello, Asha')).toBeVisible();
    console.log('fixture set up: signed in');

    await use(page);                          // the test runs here

    // Tear down: leave the cart empty for the next test.
    await page.request.delete('/api/cart');
    console.log('fixture tear down: cart emptied');
  },
});

export { expect };
```

tests/cart.spec.js

```
import { test, expect } from './fixtures.js';

test('add a mug to the cart', async ({ signedInPage: page }) => {
  await page.getByTestId('product-2').getByRole('button', { name: 'Add to cart' }).click();
  await expect(page.getByRole('status')).toHaveText('Coffee mug added to your cart');
  await expect(page.getByTestId('cart-count')).toHaveText('1');
  console.log('test: the mug is in the cart');
});
```

Run it: `npx playwright test`

Lesson: Fixtures: set up on demand

An automatic fixture that fails a test when the page throws an error

tests/fixtures.js

```
import { test as base, expect } from '@playwright/test';

export const test = base.extend({
  failOnPageErrors: [async ({ page }, use) => {
    const errors = [];
    page.on('pageerror', error => errors.push(error.message));
    await use();
    expect(errors, 'the page threw errors').toEqual([]);
  }, { auto: true }],       // every test gets it, without asking for it
});

export { expect };
```

tests/pages.spec.js

```
import { test, expect } from './fixtures.js';

test('the products page has no errors', async ({ page }) => {
```

```
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});

test('the widgets page breaks', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Break the page' }).click();
  await expect(page.getByRole('heading', { name: 'Console and errors' })).toBeVisible();
  // the test itself passes, the fixture finds the error afterwards
});
```

Run it: `npx playwright test`

Lesson: Automatic fixtures

Set up once: check the shop and create test data

playwright.config.js

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  reporter: 'list',
  use: { baseURL: 'http://localhost:4173' },
  projects: [
    { name: 'setup', testMatch: /global\.setup\.[jt]s/ },                // runs first
    { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
  ],
});
```

tests/global.setup.js

```
import { test as setup, expect } from '@playwright/test';

setup('the shop is up and has test data', async ({ request }) => {
  await expect(await request.get('/api/health')).toBeOK();
  const response = await request.post('/api/people', {
    data: { name: 'Test Customer', email: 'test@shop.example' },
  });
  expect(response.status()).toBe(201);
  console.log('setup: created person', (await response.json()).id);
});
```

tests/people.spec.js

```
import { test, expect } from '@playwright/test';

test('the test data is there', async ({ request }) => {
  const people = await (await request.get('/api/people')).json();
  console.log('test: people in the shop:', people.map(p => p.name));
  expect(people).toHaveLength(1);
});
```

Run it: `npx playwright test`

Lesson: Set up once for the whole run

TABLES

Table: when · playwright test

When	Playwright Test
before each test	test.beforeEach
after each test	test.afterEach
once before all	test.beforeAll
once after all	test.afterAll

Lesson: Before and after hooks

Table: fixture · playwright test

Fixture	Playwright Test
a fresh page	page
its browser context	context
the shared browser	browser
an API client	request
which browser	browserName
Playwright itself	playwright

Lesson: Fixtures: set up on demand

Table: scope · playwright test

Scope	Playwright Test
each test	test fixture (the default)
each worker process	{ scope: 'worker' }
the file or class	test.beforeAll
the whole run	a setup project with dependencies, or globalSetup

Lesson: Set up once for the whole run

TIPS AND TRAPS

IN REAL TEAMS Hooks run for every test in the file. Fixtures run only for tests that ask for them, and they can depend on other fixtures. As a suite grows, teams move set up from hooks into fixtures.

Lesson: Fixtures: set up on demand

4.3 Parallel runs, retries, timeouts and sharding

Four tests on two workers in each language, two tests kept apart with a lock, a flaky test that passes on retry, the five kinds of timeout with one that is too short, and a suite split into two shards.

KEY IDEAS

- **Workers** Tests at the same time. Each with its own browser.
- **Isolation** No shared data. Lock what must be shared.
- **Retries** Try again on failure. Flaky is still a bug.
- **Shards** Split across machines. Merge the reports.
- **Remember** Parallel runs only work when every test can run alone, in any order, next to any other test.

CODE

Four tests, two at a time

playwright.config.js

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  reporter: 'list',
  fullyParallel: true,              // every test can run at the same time as any other
  workers: 2,                       // two worker processes, each with its own browser
  use: { baseURL: 'http://localhost:4173' },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
});
```

tests/products.spec.js

```
import { test, expect } from '@playwright/test';
```

```
test('shows five products', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});

test('finds the lamp', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/products');
  await page.getByPlaceholder('Search products').fill('lamp');
  await expect(page.getByRole('listitem')).toHaveCount(1);
});
```

tests/orders.spec.js

```
import { test, expect } from '@playwright/test';

test('page 1 shows five orders', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/orders');
  await expect(page.getByRole('row')).toHaveCount(6);        // a header row and five orders
});

test('page 2 is the last page', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/orders');
  await page.getByRole('button', { name: 'Next' }).click();
  await expect(page.getByText('Page 2 of 2')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: Workers: tests at the same time

Two tests that must not run at the same time

tests/people.spec.js

```
import { test, expect } from '@playwright/test';

// Both tests count the people in the shop, so they must not run at the same time.
test('add Meera', { lock: 'people' }, async ({ request }) => {
  console.log('add Meera: start');
  const before = (await (await request.get('/api/people')).json()).length;
  await request.post('/api/people', { data: { name: 'Meera', email: 'meera@shop.example' } });
  const after = (await (await request.get('/api/people')).json()).length;
  expect(after).toBe(before + 1);
  console.log('add Meera: done');
});

test('add Chen', { lock: 'people' }, async ({ request }) => {
  console.log('add Chen: start');
  const before = (await (await request.get('/api/people')).json()).length;
  await request.post('/api/people', { data: { name: 'Chen', email: 'chen@shop.example' } });
  const after = (await (await request.get('/api/people')).json()).length;
  expect(after).toBe(before + 1);
  console.log('add Chen: done');
});

test('shows five products', async ({ page }) => {        // no lock, runs alongside
  console.log('products: start');
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
  console.log('products: done');
});
```

Run it: `npx playwright test`

Lesson: Parallel tests must not share data

A flaky test that passes on the second try

tests/offer.spec.js

```
import { test, expect } from '@playwright/test';

test('the offer shows up', async ({ page }, testInfo) => {
  console.log(`attempt ${testInfo.retry + 1}`);
```

```
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Load offers' }).click();
  // Flaky on purpose: the first try waits only 0.5 s, but the offer needs 1.2 s.
  const wait = testInfo.retry === 0 ? 500 : 5_000;
  await expect(page.getByTestId('offer')).toBeVisible({ timeout: wait });
});
```

Run it: `npx playwright test`

Lesson: Retries and flaky tests

Timeouts that are long enough, and one that is too short

tests/timeouts.spec.js

```
import { test, expect } from '@playwright/test';

test('waits long enough for the offer', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Load offers' }).click();
  await expect(page.getByTestId('offer')).toHaveText('Buy 2 mugs, get 1 free');   // the 5 s default is
enough
});

test('gives up too early', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Load offers' }).click();
  await expect(page.getByTestId('offer')).toBeVisible({ timeout: 1_000 });        // 1 s, but the offer
needs 1.2 s
});

test('a slow test gets more time', async ({ page }) => {
  test.slow();                                  // three times the test timeout: 90 s
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Pay now' }).click({ timeout: 3_000 });  // this click only
  await expect(page.getByText('Payment done')).toBeVisible();
});
```

Run it: `npx playwright test`

Lesson: Timeouts

Split the suite in two, as two CI machines would

tests/products.spec.js

```
import { test, expect } from '@playwright/test';

test('shows five products', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});

test('finds the lamp', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/products');
  await page.getByPlaceholder('Search products').fill('lamp');
  await expect(page.getByRole('listitem')).toHaveCount(1);
});
```

tests/orders.spec.js

```
import { test, expect } from '@playwright/test';

test('page 1 shows five orders', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/orders');
  await expect(page.getByRole('row')).toHaveCount(6);        // a header row and five orders
});

test('page 2 is the last page', async ({ page }, testInfo) => {
  console.log(`${testInfo.title} on worker ${testInfo.parallelIndex}`);
  await page.goto('/orders');
  await page.getByRole('button', { name: 'Next' }).click();
  await expect(page.getByText('Page 2 of 2')).toBeVisible();
});
```

Run it: `npx playwright test --shard=1/2 && npx playwright test --shard=2/2`

Lesson: Sharding across machines

TABLES

Table: playwright test

	Playwright Test
how	built in, worker processes
turn it on	on by default, per file
how many	workers: 4, or --workers=4
tests inside one file	in order, unless fullyParallel: true
turn it off	--workers=1

Lesson: Workers: tests at the same time

Table: playwright test

	Playwright Test
set retries	retries: 2, or --retries=2
which try is this	testInfo.retry, starts at 0
how it is reported	flaky
fail the run on flaky tests	--fail-on-flaky-tests

Lesson: Retries and flaky tests

Table: limit · default · playwright test

Limit	Default	Playwright Test
one test	30 s	timeout: 30_000, test.setTimeout(), test.slow()
one expect check	5 s	expect: { timeout: 5_000 }
one action	no limit in Playwright Test, 30 s in the libraries	use: { actionTimeout }
one navigation	no limit in Playwright Test, 30 s in the libraries	use: { navigationTimeout }
the whole run	no limit	globalTimeout

Lesson: Timeouts

TIPS AND TRAPS

REMEMBER Playwright Test runs files in parallel and the tests inside a file in order. fullyParallel: true lets every test run on any worker, which balances the load better.

Lesson: Workers: tests at the same time

COMMON MISTAKE Playwright Test also has a serial mode, test.describe.configure({ mode: 'serial' }), where the tests in a file run in order and the rest are skipped after a failure. The Playwright docs advise against it: isolated tests are faster and easier to retry.

Lesson: Parallel tests must not share data

IN REAL TEAMS Most teams turn retries on only in CI, for example retries: process.env.CI ? 2 : 0, and keep a list of flaky tests to fix. A retry also records a trace in Playwright Test when trace is on-first-retry, which is often the fastest way to find the cause.

Lesson: Retries and flaky tests

REMEMBER When a check times out, the fix is almost never a bigger number. Ask what the page was waiting for. A longer timeout is right only when the app really is that slow, such as a report that takes 20 s to build.

Lesson: Timeouts

IN REAL TEAMS Shards are balanced by test count, not by time. With fullyParallel: true, Playwright Test splits single tests, so the shards come out even. Without it, whole files are split, so keep files a similar size. pytest-split can balance by recorded durations.

Lesson: Sharding across machines

STEP 5 Network and APIs

5.1 Network: watch, mock and change requests

Logging API traffic and reading one response, mocking the products API, a 500 and a broken connection, selling out the lamp in a real answer, recording and replaying a HAR file, and mocking the live stock WebSocket.

KEY IDEAS

- **Watch** See every call. Wait for the one you need.
- **Mock** Answer with your data. No server needed.
- **Change** Real answer, one field changed. The rest stays real.
- **Replay** Record once. Same data every run.
- **Remember** Mock what you do not own or cannot control. Keep a few tests against the real API, so the mocks do not drift away from it.

CODE

Listen to the traffic and wait for one response

tests/watch.spec.js

```
import { test, expect } from '@playwright/test';

test('the products come from the API', async ({ page }) => {
  page.on('request', request => {
    if (request.url().includes('/api/')) console.log('>>', request.method(), new
URL(request.url()).pathname);
  });
  page.on('response', response => {
    if (response.url().includes('/api/')) console.log('<<', response.status(), new
URL(response.url()).pathname);
  });

  const productsCall = page.waitForResponse('**/api/products');   // start waiting before the page asks
  await page.goto('/products');
  const response = await productsCall;

  expect(response.ok()).toBeTruthy();
  const products = await response.json();
  console.log(`the API sent ${products.length} products`);
  await expect(page.getByRole('listitem')).toHaveCount(products.length);
});
```

Run it: `npx playwright test`

Lesson: Watch the traffic

Answer the products API with your own data

tests/mock.spec.js

```
import { test, expect } from '@playwright/test';

test('shows the products the API sends', async ({ page }) => {
  await page.route('**/api/products', route => route.fulfill({
```

```
    json: [
      { id: 7, name: 'Tea kettle', price: 899, stock: 4, category: 'Kitchen' },
      { id: 8, name: 'Pencil box', price: 150, stock: 0, category: 'Stationery' },
    ],
  }));

  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(2);
  await expect(page.getByTestId('product-7')).toContainText('₹899');
  await expect(page.getByTestId('product-8').getByRole('button')).toHaveText('Out of stock');
});
```

Run it: `npx playwright test`

Lesson: Mock an API

Make the API fail, and check the page copes

tests/errors.spec.js

```
import { test, expect } from '@playwright/test';

const SORRY = 'Sorry, we could not load the products. Please try again.';

test('a server error shows a friendly message', async ({ page }) => {
  await page.route('**/api/products', route => route.fulfill({ status: 500, json: { error: 'Database is
down' } }));
  await page.goto('/products');
  await expect(page.getByRole('alert')).toHaveText(SORRY);
});

test('a broken connection shows the same message', async ({ page }) => {
  await page.route('**/api/products', route => route.abort());
  await page.goto('/products');
  await expect(page.getByRole('alert')).toHaveText(SORRY);
});
```

Run it: `npx playwright test`

Lesson: Errors that are hard to cause

Keep the real answer, change one field

tests/sold-out.spec.js

```
import { test, expect } from '@playwright/test';

test('the lamp shows as sold out', async ({ page }) => {
  await page.route('**/api/products', async route => {
    const response = await route.fetch();                  // ask the real server
    const products = await response.json();
    products.find(p => p.name === 'Desk lamp').stock = 0;  // then change one field
    await route.fulfill({ response, json: products });
  });

  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);   // the other products are real
  await expect(page.getByTestId('product-3').getByRole('button')).toHaveText('Out of stock');
});
```

Run it: `npx playwright test`

Lesson: Change a real response

Record the API into a HAR file, then replay it

tests/har.spec.js

```
import { test, expect } from '@playwright/test';

test('record the products API', async ({ page }) => {
  await page.routeFromHAR('hars/products.har', { url: '**/api/products', update: true });
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});

test('replay it from the file', async ({ page }) => {
  await page.routeFromHAR('hars/products.har', { url: '**/api/products' });   // no update: answers come
from the file
```

```
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});
```

Run it: `npx playwright test && ls hars`

Lesson: Record and replay with HAR

Mock a WebSocket feed

tests/live-stock.spec.js

```
import { test, expect } from '@playwright/test';

test('shows the live stock from the server', async ({ page }) => {
  await page.goto('/widgets');
  await expect(page.getByTestId('live-stock')).toHaveText('Desk lamp: 8 left');
});

test('a mocked feed says the lamp sold out', async ({ page }) => {
  await page.routeWebSocket('**/ws/stock', ws => {          // no real server is contacted
    ws.onMessage(message => {
      if (message === 'subscribe')
        ws.send(JSON.stringify({ product: 'Desk lamp', left: 0 }));
    });
  });
  await page.goto('/widgets');
  await expect(page.getByTestId('live-stock')).toHaveText('Desk lamp: sold out');
});
```

Run it: `npx playwright test`

Lesson: WebSockets

TABLES

Table: to · playwright test

To	Playwright Test
log every request	page.on('request', ...)
log every response	page.on('response', ...)
wait for one response	page.waitForResponse('**/api/products')
wait for one request	page.waitForRequest(...)
see failed requests	page.on('requestfailed', ...)

Lesson: Watch the traffic

Table: step · what you write

Step	What you write
record	routeFromHAR(file, { url, update: true }) and run the test once
edit	change the JSON in the body file next to the HAR, if you want different data
replay	routeFromHAR(file, { url }) without update
refresh	run once more with update: true when the API changes

Lesson: Record and replay with HAR

TIPS AND TRAPS

REMEMBER

The pattern in page.route is a glob: ** matches any part of the address, including slashes, and * matches any part between two slashes. A regular expression works too. Routes added with context.route catch requests from every page of the context, including new tabs.

Lesson: Mock an API

IN REAL TEAMS Routes can also make tests faster. page.route('**/*.{png,jpg,jpeg}', route => route.abort()) stops images from loading in tests that do not need them.

Lesson: Errors that are hard to cause

REMEMBER route.continue can change the request on its way out instead: a different header, method or body. For example, route.continue({ headers: { ...route.request().headers(), "x-test": "yes" } }).

Lesson: Change a real response

COMMON MISTAKE Replay matches the address and the method exactly, and for POST also the body. A request that matches nothing in the file is aborted, unless you pass notFound: fallback.

Lesson: Record and replay with HAR

5.2 API testing and signing in once

A full create, read, update and delete test of the people API with a validation error, a token sent in a header, a cart filled through the API and checked in the page, and a saved sign in reused by every test.

KEY IDEAS

- **API tests** No browser at all. Status, data, errors.
- **Tokens** Proof of who calls. Keep them in secrets.
- **Mix** API for set up. Page for the check.
- **Sign in once** Save the state. Every test starts signed in.
- **Remember** Use the page for what the user sees. Use the API for everything you only need to get ready.

CODE

Create, read, update and delete through the API

tests/people-api.spec.js

```
import { test, expect } from '@playwright/test';

test('create, read, update and delete a person', async ({ request }) => {
  const created = await request.post('/api/people', { data: { name: 'Kiran', email: 'kiran@shop.example' }
});
  expect(created.status()).toBe(201);
  const kiran = await created.json();
  expect(created.headers()['location']).toBe(`/api/people/${kiran.id}`);

  const read = await request.get(`/api/people/${kiran.id}`);
  await expect(read).toBeOK();
  expect(await read.json()).toEqual({ id: kiran.id, name: 'Kiran', email: 'kiran@shop.example', city: null
});

  const updated = await request.patch(`/api/people/${kiran.id}`, { data: { city: 'Pune' } });
  expect((await updated.json()).city).toBe('Pune');

  expect((await request.delete(`/api/people/${kiran.id}`)).status()).toBe(204);
  expect((await request.get(`/api/people/${kiran.id}`)).status()).toBe(404);
});

test('a bad email is refused', async ({ request }) => {
  const response = await request.post('/api/people', { data: { name: 'Nobody', email: 'not-an-email' } });
  expect(response.status()).toBe(422);
  expect(await response.json()).toEqual({ error: 'name and a valid email are required' });
});
```

Run it: `npx playwright test`

Lesson: API tests without a browser

Sign in through the API and send the token

tests/token.spec.js

```
import { test, expect } from '@playwright/test';
```

```
test('the token opens the cart', async ({ request, playwright }) => {
  expect((await request.get('/api/cart')).status()).toBe(401);      // nobody is signed in yet

  const login = await request.post('/api/login', { data: { email: 'asha@shop.example', password: 's3cret' }
});
  const { token, name } = await login.json();
  console.log(`signed in as ${name}`);

  // A fresh client that sends the token with every call, and has no cookies.
  const asha = await playwright.request.newContext({
    baseURL: 'http://localhost:4173',
    extraHTTPHeaders: { Authorization: `Bearer ${token}` },
  });
  await asha.post('/api/cart', { data: { productId: 2, qty: 2 } });
  const cart = await (await asha.get('/api/cart')).json();
  console.log(cart);
  expect(cart.count).toBe(2);
  expect(cart.total).toBe(398);
  await asha.dispose();
});
```

Run it: `npx playwright test`

Lesson: Tokens and headers

Arrange through the API, check through the page

tests/cart-setup.spec.js

```
import { test, expect } from '@playwright/test';

test('a lamp added through the API shows in the cart page', async ({ page }) => {
  // Sign in through the API. page.request shares cookies with the page.
  const login = await page.request.post('/api/login', { data: { email: 'asha@shop.example', password:
's3cret' } });
  const { token } = await login.json();
  await page.addInitScript(t => localStorage.setItem('token', t), token);   // the shop keeps its token here

  await page.request.post('/api/cart', { data: { productId: 3 } });         // arrange the cart without
clicking

  await page.goto('/cart');
  await expect(page.getByText('Hello, Asha')).toBeVisible();
  await expect(page.getByRole('row', { name: /Desk lamp/ })).toContainText('₹1,299');
  await expect(page.getByTestId('cart-count')).toHaveText('1');
});
```

Run it: `npx playwright test`

Lesson: API for set up, page for checks

Sign in once, save the state, start every test signed in

playwright.config.js

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  reporter: 'list',
  use: { baseURL: 'http://localhost:4173' },
  projects: [
    { name: 'setup', testMatch: /auth\.setup\.[jt]s/ },
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/asha.json' },   // every test
starts signed in
      dependencies: ['setup'],
    },
  ],
});
```

tests/auth.setup.js

```
import { test as setup, expect } from '@playwright/test';

setup('sign in as Asha', async ({ page }) => {
  await page.goto('/login');
```

```
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByText('Hello, Asha')).toBeVisible();          // wait until signing in has finished
  await page.context().storageState({ path: 'playwright/.auth/asha.json' });
});
```

tests/signed-in.spec.js

```
import { test, expect } from '@playwright/test';

test('the products page greets Asha', async ({ page }) => {
  await page.goto('/products');                       // no sign in steps here
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});

test('the cart is open to her', async ({ page }) => {
  await page.goto('/cart');
  await expect(page.getByRole('heading', { name: 'Your cart' })).toBeVisible();
  await expect(page).toHaveURL(/\/cart$/);            // not sent to /login
});
```

Run it: `npx playwright test`

Lesson: Sign in once for the whole suite

TABLES

Table: playwright test

	Playwright Test
the client	the request fixture
send JSON	request.post(url, { data })
check status 200 to 299	await expect(response).toBeOK()
read JSON	await response.json()

Lesson: API tests without a browser

Table: need · approach

Need	Approach
all tests can share one account	one setup step and one state file, as above
tests change data of the account	one account per worker, named with the worker index
two roles, such as customer and admin	two state files, asha.json and ravi.json, and a project or fixture for each
the app keeps its session in sessionStorage	storageState does not save sessionStorage, so save it yourself and restore it with addInitScript

Lesson: Sign in once for the whole suite

TIPS AND TRAPS

COMMON MISTAKE Keep real passwords and tokens out of test code. Read them from environment variables, such as process.env.SHOP_PASSWORD, os.environ["SHOP_PASSWORD"] or System.getenv("SHOP_PASSWORD"), and set them as secrets in CI. The framework chapter shows a .env file for local runs.

Lesson: Tokens and headers

REMEMBER page.request (and context.request) share cookies with the browser context. A sign in through page.request signs in the page too, for apps that use cookies. The shop also keeps a token in local storage, which is why the example adds it with addInitScript.

Lesson: API for set up, page for checks

COMMON MISTAKE The state file holds a live session that anyone could use. Add playwright/.auth to .gitignore, and let each run create it fresh.

Lesson: Sign in once for the whole suite

STEP 6 Debugging and quality

6.1 Debugging failed tests: errors, traces, reports and the Inspector

A failed check read line by line in four languages, a failed click that leaves a screenshot, a video and a trace, a trace with named steps, verbose logs, and real screenshots of the trace viewer, the HTML report, UI mode and the Inspector.

KEY IDEAS

- **Error** Expected and received. Read it first.
- **Evidence** Screenshot, video, trace. Kept on failure.
- **Trace** Every action replayed. DOM, network, console.
- **Inspector** Run it yourself. One step at a time.
- **Remember** Read the error, then the trace, and only then run the test by hand. Most failures are solved at the first step.

CODE

A failed check, and what the runner prints

tests/greeting.spec.js

```
import { test, expect } from '@playwright/test';

test('greets Asha after sign in', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.locator('#hello')).toHaveText('Hello, Asha!');   // the page says Hello, Asha
});
```

Run it: `npx playwright test`

Lesson: Read the error first

A screenshot, a video and a trace when a test fails

playwright.config.js

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  reporter: 'list',
  use: {
    baseURL: 'http://localhost:4173',
    screenshot: 'only-on-failure',   // a picture of the page when a test fails
    video: 'retain-on-failure',      // a video of every test, kept only when it fails
    trace: 'retain-on-failure',      // a trace of every test, kept only when it fails
  },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
});
```

tests/cart.spec.js

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('asha@shop.example');
  await page.getByLabel('Password').fill('s3cret');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.getByRole('listitem')).toHaveCount(5);
});
```

```
test('add the lamp to the cart', async ({ page }) => {
  await page.getByTestId('product-3').getByRole('button', { name: 'Add to cart' }).click();
  await expect(page.getByTestId('cart-count')).toHaveText('1');
});

test('add the notebook to the cart', async ({ page }) => {
  // the Notebook is out of stock, so this button never appears
  await page.getByTestId('product-4').getByRole('button', { name: 'Add to cart' }).click({ timeout: 3000 });
});
```

Run it: `npx playwright test ; find test-results -type f | sort`

Lesson: Screenshots, videos and traces on failure

A trace with named steps

tests/mug.spec.js

```
import { test, expect } from '@playwright/test';

test('buy a mug', async ({ page }) => {
  await test.step('sign in', async () => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('asha@shop.example');
    await page.getByLabel('Password').fill('s3cret');
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.locator('#hello')).toHaveText('Hello, Asha');
  });
  await test.step('add the mug', async () => {
    await page.getByTestId('product-2').getByRole('button', { name: 'Add to cart' }).click();
    await expect(page.getByTestId('cart-count')).toHaveText('1');
  });
});
```

Run it: `npx playwright test && find test-results -name trace.zip`

Lesson: The trace viewer

Every Playwright call, written to the terminal

tests/lamp.spec.js

```
import { test, expect } from '@playwright/test';

test('find the lamp', async ({ page }) => {
  await page.goto('/products');
  await page.getByPlaceholder('Search products').fill('lamp');
  await expect(page.getByRole('listitem')).toHaveCount(1);
});
```

Run it: `DEBUG=pw:api npx playwright test`

Lesson: Verbose logs

TABLES

Table: part · what it tells you

Part	What it tells you
the locator	which element the check or action looked for
expected and received	what the test wanted and what the page had, side by side
the timeout	how long Playwright kept trying before it gave up
the call log	each try: did the locator find an element, was it visible, enabled and stable, what did it contain

Lesson: Read the error first

Table: the error says · it usually means

The error says	It usually means
received a different text or count	the page changed, or the expected value in the test is out of date
element(s) not found, or waiting for locator	the locator is wrong, a step before it did not happen, or the page did not load
strict mode violation, resolved to 2 elements	the locator is too loose, so narrow it with a role, a name or a parent
element is not enabled, or is not visible	the app has not finished something, such as loading data or validating a form
Test timeout of 30000ms exceeded	one call waited for something that never happened, and the call log names it

Lesson: Read the error first

Table: setting · playwright test

Setting	Playwright Test
screenshot	screenshot: 'only-on-failure'
video	video: 'retain-on-failure'
trace	trace: 'retain-on-failure'
where	test-results/
other values	'on', 'off', 'on-first-retry', 'retain-on-first-failure'

Lesson: Screenshots, videos and traces on failure

Table: open a trace · command

Open a trace	Command
JavaScript and TypeScript	npx playwright show-trace test-results/.../trace.zip
any language, no install	drag the file onto trace.playwright.dev, which reads it in your browser

Lesson: The trace viewer

Table: javascript and typescript

	JavaScript and TypeScript
open the Inspector	npx playwright test --debug
one test only	npx playwright test cart.spec.ts:12 --debug
stop at a line	await page.pause()
see the source	shown by itself
watch slowly	launchOptions: { slowMo: 500 }

Lesson: Step through a test with the Inspector

Table: variable · what it prints

Variable	What it prints
DEBUG=pw:api	every Playwright call and the steps inside it
DEBUG=pw:browser	what the browser process prints, useful when it will not start
DEBUG=pw:protocol	every message between Playwright and the browser, very long
PWDEBUG=console	a playwright object in the browser DevTools, to try locators in the console

Lesson: Verbose logs

TIPS AND TRAPS

REMEMBER Read the received value carefully. A missing full stop, an extra space or a different currency sign is the most common cause of a red check, and the error shows it to you directly.

Lesson: Read the error first

IN REAL TEAMS Teams often use trace: 'on-first-retry' in CI. A first run that fails is retried with tracing on, so the trace is recorded only for tests that failed at least once.

Lesson: Screenshots, videos and traces on failure

REMEMBER Inside the trace viewer, click an action and then use the Before and After tabs to compare the page. The Pick locator button lets you try a new locator on the saved page.

Lesson: The trace viewer

COMMON MISTAKE UI mode, the HTML report and the VS Code extension are part of Playwright Test, so they are there for JavaScript and TypeScript. The reporting chapter builds each of them.

Lesson: The HTML report and UI mode

COMMON MISTAKE Remove page.pause() before you commit. It needs a headed browser and is only useful while you debug.

Lesson: Step through a test with the Inspector

REMEMBER Browser console messages and page errors are a different stream. Collect them with page.on("console") and page.on("pageerror"), as the fixtures chapter does, or read them in the Console tab of the trace.

Lesson: Verbose logs

6.2 Visual, accessibility and time checks

A screenshot comparison that catches a style bug with a real diff picture, aria snapshots of the menu and a product card, an axe scan that finds a missing alt text and low contrast, and a sale date and an idle sign out tested with a fake clock.

KEY IDEAS

- **Screenshot** How it looks. Baseline and diff.
- **Aria snapshot** How it reads. Every language.
- **axe scan** Who can use it. WCAG rules.
- **Fake clock** Any date, any delay. No waiting.
- **Remember** Screenshots check pixels, aria snapshots check structure, axe checks access, and the clock checks time. Use each for the mistakes it is good at.

CODE

Compare the page with a saved picture

tests/look.spec.js

```
import { test, expect } from '@playwright/test';

test('the products page looks right', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(5);
  await expect(page).toHaveScreenshot('products.png', { fullPage: true });
});

test('the lamp card looks right', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByTestId('product-3')).toHaveScreenshot('lamp-card.png');
});

test('a button colour changed by mistake', async ({ page }) => {
  await page.goto('/products');
  await page.addStyleTag({ content: '.card button { background: #b91c1c; }' });   // a style bug, on purpose
  await expect(page.getByTestId('product-3')).toHaveScreenshot('lamp-card.png');
});
```

Run it: `npx playwright test -g "looks right" --update-snapshots $ npx playwright test $ find tests test-results -name "*.png"`

Lesson: Visual comparisons

Check the structure of the page with an aria snapshot

tests/structure.spec.js

```
import { test, expect } from '@playwright/test';

test('the main menu has four links', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('navigation', { name: 'Main' })).toMatchAriaSnapshot(`
    - navigation "Main":
      - link "Products"
      - link "Feedback"
      - link "Widgets"
      - link "Orders"
  `);
});

test('the notebook card says it is out of stock', async ({ page }) => {
  await page.goto('/products');
  console.log(await page.getByTestId('product-4').ariaSnapshot());   // print it once, then paste it below
  await expect(page.getByTestId('product-4')).toMatchAriaSnapshot(`
    - listitem:
      - heading "Notebook" [level=3]
      - text: Stationery ₹60
      - button "Out of stock" [disabled]
  `);
});
```

Run it: `npx playwright test`

Lesson: Aria snapshots

Scan a page for accessibility problems with axe

tests/offers.spec.js

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test('the offers page has no accessibility problems', async ({ page }) => {
  await page.goto('/a11y');
  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa'])          // the WCAG 2 A and AA rules
    .analyze();
  for (const v of results.violations) {
    console.log(`${v.id} (${v.impact}): ${v.nodes.length} element(s). ${v.help}`);
  }
  expect(results.violations.map(v => v.id)).toEqual([]);
});
```

Run it: `npx playwright test`

Lesson: Accessibility scans with axe

Control dates and timers with a fake clock

tests/clock.spec.js

```
import { test, expect } from '@playwright/test';

test('the sale is not live the day before', async ({ page }) => {
  await page.clock.setFixedTime(new Date('2026-12-24T12:00:00Z'));
  await page.goto('/clock');
  await expect(page.getByRole('status')).toHaveText('The Christmas sale is coming soon');
});

test('the sale goes live on Christmas Day', async ({ page }) => {
  await page.clock.setFixedTime(new Date('2026-12-25T12:00:00Z'));
  await page.goto('/clock');
  await expect(page.getByRole('status')).toHaveText('The Christmas sale is live');
});

test('an idle user is signed out after 5 minutes', async ({ page }) => {
  await page.clock.install({ time: new Date('2026-12-24T10:00:00Z') });
  await page.goto('/clock');
  await page.clock.runFor('05:00');                    // 5 minutes of timers, in a moment
```

```
  await expect(page.getByRole('alert')).toHaveText('You were signed out');
});
```

Run it: `npx playwright test`

Lesson: Fake time with page.clock

TABLES

Table: option · what it does

Option	What it does
--update-snapshots	saves new baselines instead of comparing, after a change you meant to make
maxDiffPixels, maxDiffPixelRatio	how many pixels, or what share of them, may differ
threshold	how different one pixel may be before it counts, from 0 to 1, default 0.2
mask: [locator]	covers parts that change every run, such as a date or an advert, with a box
animations: "disabled"	stops CSS animations so the picture is the same every time (the default)
stylePath	adds a style sheet only while taking the picture, for example to hide a clock

Lesson: Visual comparisons

Table: playwright test

	Playwright Test
print a snapshot	await locator.ariaSnapshot()
check a snapshot	toMatchAriaSnapshot(`...`)
keep it in a file	toMatchAriaSnapshot({ name: "menu.aria.yml" })

Lesson: Aria snapshots

Table: what axe found · why it matters · the fix

What axe found	Why it matters	The fix
image-alt: the lamp image has no alt text	a screen reader cannot say what the picture shows	add alt="Desk lamp"
color-contrast: light blue text on a light background	people with low vision cannot read it	use a darker colour, with a contrast ratio of at least 4.5 to 1

Lesson: Accessibility scans with axe

Table: package · language · made by

Package	Language	Made by
@axe-core/playwright	JavaScript and TypeScript	Deque
com.deque.html.axe-core:playwright	Java	Deque
axe-playwright-python	Python	community

Lesson: Accessibility scans with axe

Table: method · what it does

Method	What it does
setFixedTime(time)	Date.now() always returns this time. Timers still run normally.
install({ time })	replaces Date and every timer. Call it before the page loads.
runFor("05:00")	moves forward, firing every timer that falls due on the way
fastForward("05:00")	jumps forward and fires due timers at most once, like a laptop waking from sleep
pauseAt(time), resume()	stops time at a moment to check the page, then lets it run again
setSystemTime(time)	changes the date without firing timers, like a user changing the system clock

Lesson: Fake time with page.clock

TIPS AND TRAPS

IN REAL TEAMS Baselines depend on the operating system and the browser, which is why the file name ends in chromium-linux. Make them on the same system that runs the tests, usually the CI image or the Playwright Docker image, and commit them to the repository.
Lesson: Visual comparisons

COMMON MISTAKE toHaveScreenshot is part of Playwright Test. Teams add a library for it or use a visual testing service. The aria snapshot below works in every language.
Lesson: Visual comparisons

REMEMBER Codegen can record aria snapshots too: use the Assert snapshot button in the toolbar while recording, and the snapshot of the element you click is added to the test.
Lesson: Aria snapshots

COMMON MISTAKE An automatic scan finds only part of the problems. It cannot tell whether alt text is useful or whether the tab order makes sense. Keep a few manual checks with a keyboard and a screen reader, and use the scan to stop the easy mistakes from coming back.
Lesson: Accessibility scans with axe

IN REAL TEAMS An old page may have many known problems. Scan with .exclude() for the parts you will fix later, or compare the list of violations with a saved list, so new problems still fail the test.
Lesson: Accessibility scans with axe

REMEMBER Use setFixedTime when the page only reads the date, and install with runFor when it uses setTimeout or setInterval. Either way the test runs in milliseconds and gives the same result on every day of the year.
Lesson: Fake time with page.clock

STEP 7 Framework design

7.1 Page objects and components

A LoginPage, a ProductsPage built from Header and ProductCard components, and fixtures that hand out signed in page objects, each run for real in four languages.

KEY IDEAS

- **Page object** One class per screen. Locators in one place.
- **Component** Parts that repeat. Root locator first.
- **Fixtures** Hand them out. No set up in tests.
- **Checks** Stay in the test. Easy to read.
- **Remember** The test says what, the page object says how. A change to the screen is fixed in one class.

CODE

Step 1: one page object for the sign in page

pages/LoginPage.js

```
// Everything a test needs to know about the sign in page, in one place
export class LoginPage {
  constructor(page) {
    this.page = page;
    this.email = page.getByLabel('Email');
    this.password = page.getByLabel('Password');
    this.signInButton = page.getByRole('button', { name: 'Sign in' });
    this.error = page.getByRole('alert');
  }

  async goto() {
    await this.page.goto('/login');
  }

  async signIn(email, password) {
    await this.email.fill(email);
    await this.password.fill(password);
    await this.signInButton.click();
  }
}
```

tests/login.spec.js

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage.js';

test('Asha can sign in', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.signIn('asha@shop.example', 's3cret');
  await expect(page.getByText('Hello, Asha')).toBeVisible();
});

test('a wrong password is refused', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.signIn('asha@shop.example', 'wrong');
  await expect(loginPage.error).toHaveText('Wrong email or password');
});
```

Run it: `npx playwright test`

Lesson: Why page objects

Step 2: components for parts that repeat

components/ProductCard.js

```
// One product card. Every locator starts from the card, so it never finds another product's button.
export class ProductCard {
  constructor(root) {
    this.root = root;
    this.price = root.locator('.price');
    this.button = root.getByRole('button');
  }

  async addToCart() {
    await this.root.getByRole('button', { name: 'Add to cart' }).click();
  }
}
```

components/Header.js

```
// The header is on every page, so it is a component, not a page
export class Header {
  constructor(page) {
    const root = page.getByRole('banner');
    this.greeting = root.locator('#hello');
    this.cartCount = root.getByTestId('cart-count');
    this.signOut = root.getByRole('link', { name: 'Sign out' });
```

```
  }
}
```

pages/ProductsPage.js

```
import { Header } from '../components/Header.js';
import { ProductCard } from '../components/ProductCard.js';

export class ProductsPage {
  constructor(page) {
    this.page = page;
    this.header = new Header(page);
    this.search = page.getByPlaceholder('Search products');
    this.cards = page.getByRole('listitem');
  }

  async goto() {
    await this.page.goto('/products');
  }

  card(name) {
    return new ProductCard(this.cards.filter({ has: this.page.getByRole('heading', { name }) }));
  }
}
```

tests/cart.spec.js

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage.js';
import { ProductsPage } from '../pages/ProductsPage.js';

test('Asha adds the lamp and the mug', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.signIn('asha@shop.example', 's3cret');

  const products = new ProductsPage(page);
  await expect(products.header.greeting).toHaveText('Hello, Asha');
  await products.card('Desk lamp').addToCart();
  await expect(products.header.cartCount).toHaveText('1');
  await products.card('Coffee mug').addToCart();
  await expect(products.header.cartCount).toHaveText('2');
});

test('the notebook is out of stock', async ({ page }) => {
  const products = new ProductsPage(page);
  await products.goto();
  const notebook = products.card('Notebook');
  await expect(notebook.price).toHaveText('₹60');
  await expect(notebook.button).toHaveText('Out of stock');
  await expect(notebook.button).toBeDisabled();
});
```

Run it: `npx playwright test`

Lesson: Components for repeated parts

Step 3: page objects as fixtures

fixtures/pages.js

```
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage.js';
import { ProductsPage } from '../pages/ProductsPage.js';

// Tests ask for loginPage or productsPage by name, like they ask for page
export const test = base.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  productsPage: async ({ page, loginPage }, use) => {
    await loginPage.goto();
    await loginPage.signIn('asha@shop.example', 's3cret');   // productsPage always starts signed in
    await use(new ProductsPage(page));
  },
});
```

```
export { expect } from '@playwright/test';
```

tests/mug.spec.js

```
import { test, expect } from '../fixtures/pages.js';

test('the mug costs 199 rupees and can be added', async ({ productsPage }) => {
  const mug = productsPage.card('Coffee mug');
  await expect(mug.price).toHaveText('₹199');
  await mug.addToCart();
  await expect(productsPage.header.cartCount).toHaveText('1');
});

test('the search finds one lamp', async ({ productsPage }) => {
  await productsPage.search.fill('lamp');
  await expect(productsPage.cards).toHaveCount(1);
});
```

Run it: `npx playwright test`

Lesson: Handing out page objects

TABLES

Table: without page objects · with page objects

Without page objects	With page objects
locators copied into every test	each locator written once
a UI change breaks many files	a UI change is fixed in one class
tests read as clicks and fills	tests read as steps: signIn, addToCart
new team members copy old tests	new tests reuse a small, named set of methods

Lesson: Why page objects

Table: playwright test

	Playwright Test
where page objects are created	test.extend in a fixtures file
how a test gets one	by name: { productsPage }
only built when used	yes

Lesson: Handing out page objects

Table: rule · why

Rule	Why
one class per screen or component	easy to find, small enough to read
locators by role, label or test id	they survive style changes, as the locators chapter showed
methods are user actions: signIn, addToCart	tests read like steps, not clicks
checks live in tests	the test shows what it proves
no sleeps and no waits for timeouts	Playwright actions and checks already wait
no test data inside the class	pass it in, so one page object serves many tests
a method returns the next page object when the screen changes	signIn() can return ProductsPage, so the flow is clear

Lesson: Rules teams follow

TIPS AND TRAPS

REMEMBER Return locators, or small components, and leave the checks in the test. A test that says expect(notebook.button).toBeDisabled() shows what it proves. A page object method called checkNotebook() hides it.

Lesson: Components for repeated parts

IN REAL TEAMS Large suites sometimes add one more layer of business steps, such as checkout(user, items), built from page objects. Add it when you notice the same five page object calls in many tests, not before.

Lesson: Rules teams follow

7.2 Framework design, step by step

The same framework built and run in TypeScript, JavaScript, Python and Java: the runner finds the tests, the full suite passes, the smoke set runs alone, and every file is shown with why it is there.

KEY IDEAS

- **Settings** From the environment. No secrets in Git.
- **Layers** One job each. Data, API, pages.
- **Fixtures** Fresh session per test. Safe in parallel.
- **Tests** By feature, tagged. Steps and checks only.
- **Remember** Framework code and tests live apart. Tests say what, the framework says how, and the environment says where.

CODE

Step 1: the project, its settings and its runner config

package.json

```
{
  "name": "shop-tests",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "playwright test",
    "test:smoke": "playwright test --grep @smoke",
    "report": "playwright show-report"
  },
  "devDependencies": {
    "@playwright/test": "1.63.0",
    "dotenv": "18.0.9"
  }
}
```

playwright.config.js

```
import 'dotenv/config';
import { defineConfig, devices } from '@playwright/test';
import { env } from './src/config/env.js';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,                // a forgotten test.only fails the CI run
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: process.env.CI
    ? [['list'], ['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }]]
    : [['list'], ['html', { open: 'never' }]],
  use: {
    baseURL: env.baseURL,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
```

```
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    // { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    // { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

src/config/env.js

```
// Settings that change between machines (a laptop, CI, staging) live here and nowhere else
export const env = {
  baseURL: process.env.BASE_URL ?? 'http://localhost:4173',
};
```

Run it: `npx playwright test --list`

Lesson: Step 1: project, settings and runner config

Step 2: data, API helpers, components, page objects and fixtures

src/data/users.js

```
// Test accounts. Real projects read passwords from CI secrets, never from the repository.
export const users = {
  asha: { email: 'asha@shop.example', password: process.env.ASHA_PASSWORD || 's3cret', name: 'Asha' },
  ravi: { email: 'ravi@shop.example', password: process.env.RAVI_PASSWORD || 'admin123', name: 'Ravi' },
};
```

src/data/products.js

```
// The products the tests use, with the ids the API expects
export const products = {
  gelPen: { id: 1, name: 'Gel pen', price: '₹20' },
  coffeeMug: { id: 2, name: 'Coffee mug', price: '₹199' },
  deskLamp: { id: 3, name: 'Desk lamp', price: '₹1,299' },
  notebook: { id: 4, name: 'Notebook', price: '₹60' },
  waterBottle: { id: 5, name: 'Water bottle', price: '₹349' },
};
```

src/api/ShopApi.js

```
// Fast set up through the API, so UI tests only click what they are testing
export class ShopApi {
  constructor(request) {
    this.request = request;
  }

  async signIn(user) {
    const response = await this.request.post('/api/login', { data: { email: user.email, password:
user.password } });
    if (!response.ok()) throw new Error(`Sign in failed for ${user.email}: ${response.status()}`);
    return (await response.json()).token;
  }

  async addToCart(token, productId, qty = 1) {
    const response = await this.request.post('/api/cart', {
      data: { productId, qty },
      headers: { Authorization: `Bearer ${token}` },
    });
    if (!response.ok()) throw new Error(`Could not add product ${productId}: ${response.status()}`);
  }
}
```

src/components/Header.js

```
// The header is on every page, so it is a component that every page object shares
export class Header {
  constructor(page) {
    const root = page.getByRole('banner');
    this.greeting = root.locator('#hello');
    this.cartCount = root.getByTestId('cart-count');
    this.signOut = root.getByRole('link', { name: 'Sign out' });
  }
}
```

src/components/ProductCard.js

```
// One product card. Every locator starts from the card, so it never finds another product's button.
export class ProductCard {
```

```
  constructor(root) {
    this.root = root;
    this.price = root.locator('.price');
    this.button = root.getByRole('button');
  }

  async addToCart() {
    await this.root.getByRole('button', { name: 'Add to cart' }).click();
  }
}
```

src/pages/BasePage.js

```
import { Header } from '../components/Header.js';

// What every page shares: the browser page, its address and the header
export class BasePage {
  constructor(page, path) {
    this.page = page;
    this.path = path;
    this.header = new Header(page);
  }

  async open() {
    await this.page.goto(this.path);
  }
}
```

src/pages/LoginPage.js

```
import { BasePage } from './BasePage.js';

export class LoginPage extends BasePage {
  constructor(page) {
    super(page, '/login');
    this.email = page.getByLabel('Email');
    this.password = page.getByLabel('Password');
    this.signInButton = page.getByRole('button', { name: 'Sign in' });
    this.error = page.getByRole('alert');
  }

  async signIn(user) {
    await this.email.fill(user.email);
    await this.password.fill(user.password);
    await this.signInButton.click();
  }
}
```

src/pages/ProductsPage.js

```
import { ProductCard } from '../components/ProductCard.js';
import { BasePage } from './BasePage.js';

export class ProductsPage extends BasePage {
  constructor(page) {
    super(page, '/products');
    this.search = page.getByPlaceholder('Search products');
    this.cards = page.getByRole('list', { name: 'Products' }).getByRole('listitem');
    this.noResults = page.getByText('No products match your search');
  }

  card(name) {
    return new ProductCard(this.cards.filter({ has: this.page.getByRole('heading', { name }) }));
  }

  async searchFor(text) {
    await this.search.fill(text);
  }
}
```

src/pages/CartPage.js

```
import { BasePage } from './BasePage.js';

export class CartPage extends BasePage {
  constructor(page) {
```

```
    super(page, '/cart');
    this.items = page.getByRole('table', { name: 'Cart items' }).locator('tbody tr');
    this.total = page.getByTestId('total');
    this.placeOrderButton = page.getByRole('button', { name: 'Place order' });
    this.status = page.getByRole('status');
  }

  async placeOrder() {
    this.page.once('dialog', dialog => dialog.accept());   // the shop asks to confirm the order
    await this.placeOrderButton.click();
  }
}
```

src/fixtures/index.js

```
import { test as base, expect } from '@playwright/test';
import { ShopApi } from '../api/ShopApi.js';
import { users } from '../data/users.js';
import { CartPage } from '../pages/CartPage.js';
import { LoginPage } from '../pages/LoginPage.js';
import { ProductsPage } from '../pages/ProductsPage.js';

// Every test file imports test and expect from here instead of from @playwright/test
export const test = base.extend({
  api: async ({ request }, use) => {
    await use(new ShopApi(request));
  },
  // A fresh session made through the API: fast, and no two tests share a cart
  session: async ({ api, page }, use) => {
    const token = await api.signIn(users.asha);
    await page.addInitScript(value => window.localStorage.setItem('token', value), token);
    await use({ user: users.asha, token });
  },
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  productsPage: async ({ page }, use) => {
    await use(new ProductsPage(page));
  },
  cartPage: async ({ page }, use) => {
    await use(new CartPage(page));
  },
});

export { expect };
```

Run it: `npx playwright test`

Lesson: Step 2: data, API helpers, page objects and fixtures

Step 3: tests by feature, tagged so a smaller set can run

tests/smoke/sign-in.spec.js

```
import { test, expect } from '../../src/fixtures/index.js';
import { users } from '../../src/data/users.js';

test.describe('sign in', { tag: '@smoke' }, () => {
  test('Asha signs in with the right password', async ({ loginPage, productsPage }) => {
    await loginPage.open();
    await loginPage.signIn(users.asha);
    await expect(productsPage.header.greeting).toHaveText('Hello, Asha');
  });

  test('a wrong password shows an error', async ({ loginPage }) => {
    await loginPage.open();
    await loginPage.signIn({ ...users.asha, password: 'wrong' });
    await expect(loginPage.error).toHaveText('Wrong email or password');
  });
});
```

tests/products/search.spec.js

```
import { test, expect } from '../../src/fixtures/index.js';
import { products } from '../../src/data/products.js';

test.describe('product search', () => {
```

```
  test('finds the lamp by name', async ({ productsPage }) => {
    await productsPage.open();
    await productsPage.searchFor('lamp');
    await expect(productsPage.cards).toHaveCount(1);
    await expect(productsPage.card(products.deskLamp.name).price).toHaveText(products.deskLamp.price);
  });

  test('says so when nothing matches', async ({ productsPage }) => {
    await productsPage.open();
    await productsPage.searchFor('piano');
    await expect(productsPage.noResults).toBeVisible();
  });

  test('the notebook cannot be added', async ({ productsPage }) => {
    await productsPage.open();
    await expect(productsPage.card(products.notebook.name).button).toBeDisabled();
  });
});
```

tests/cart/add-to-cart.spec.js

```
import { test, expect } from '../../src/fixtures/index.js';
import { products } from '../../src/data/products.js';

test.describe('cart', { tag: '@cart' }, () => {
  test('adds two products from their cards', async ({ session, productsPage, cartPage }) => {
    await productsPage.open();
    await expect(productsPage.header.greeting).toHaveText(`Hello, ${session.user.name}`);
    await productsPage.card(products.deskLamp.name).addToCart();
    await productsPage.card(products.coffeeMug.name).addToCart();
    await expect(productsPage.header.cartCount).toHaveText('2');

    await cartPage.open();
    await expect(cartPage.items).toHaveCount(2);
    await expect(cartPage.total).toHaveText('₹1,498');
  });
});
```

tests/cart/checkout.spec.js

```
import { test, expect } from '../../src/fixtures/index.js';
import { products } from '../../src/data/products.js';

test.describe('checkout', { tag: ['@cart', '@smoke'] }, () => {
  test('places an order for a cart filled through the API', async ({ session, api, cartPage }) => {
    await api.addToCart(session.token, products.waterBottle.id, 2);   // arrange through the API, test the
UI
    await cartPage.open();
    await expect(cartPage.total).toHaveText('₹698');

    await cartPage.placeOrder();
    await expect(cartPage.status).toHaveText(/Order #\d+ placed/);
    await expect(cartPage.header.cartCount).toHaveText('0');
  });
});
```

Run it: `npx playwright test --grep @smoke`

Lesson: Step 3: tests by feature, with tags

TABLES

Table: folder · typescript and javascript

Folder	TypeScript and JavaScript
framework code	src/
settings	src/config/env.ts
page objects	src/pages/
fixtures	src/fixtures/index.ts
tests	tests/<feature>/*.spec.ts
runner settings	playwright.config.ts

Lesson: What a framework is, and the finished result

Table: rule · why

Rule	Why
settings come from the environment, with a local default	the same code runs on a laptop, in CI and against staging
secrets never go in the repository	passwords live in CI secrets and in a local .env that Git ignores
exact versions of every package	every machine, and every CI run, uses the same Playwright

Lesson: Step 1: project, settings and runner config

Table: layer · its one job · in this framework

Layer	Its one job	In this framework
data	test users and products	users, products
API helper	fast set up without the browser	ShopApi: signIn, addToCart
components	parts on many pages	Header, ProductCard
page objects	one screen each	BasePage, LoginPage, ProductsPage, CartPage
fixtures	hand objects and a session to each test	test.extend, conftest.py, BaseTest

Lesson: Step 2: data, API helpers, page objects and fixtures

Table: playwright test

	Playwright Test
mark a test	{ tag: '@smoke' }
run only those	npx playwright test --grep @smoke
leave them out	--grep-invert @slow
register the names	not needed

Lesson: Step 3: tests by feature, with tags

Table: when you need · add · covered in

When you need	Add	Covered in
several environments	one .env per environment, or CI variables per job	this chapter, step 1
many signed in roles	a setup step that saves storage state per role	API testing
a run on every pull request	a CI workflow with shards and a merged report	CI/CD
reports for managers and testers	Allure, pytest-html or ExtentReports next to the built-in reports	Reporting labs
visual and accessibility checks	toHaveScreenshot, aria snapshots and axe in their own folder	Visual, accessibility and time
lots of varied test data	small factory functions that build users and orders with unique names	Parallel runs
consistent code	a linter, a formatter and code review rules	your team

Lesson: How the framework grows

TIPS AND TRAPS

IN REAL TEAMS Commit a .env.example with the names of the settings and no values. A new team member copies it to .env and fills it in. The real .env is listed in .gitignore.

Lesson: Step 1: project, settings and runner config

REMEMBER The session fixture signs in through the API and puts the token in the page before it loads. Each test gets its own session, so its cart is its own, and parallel tests cannot disturb each other. This is the isolation rule from the parallel runs chapter, built into the framework.

Lesson: Step 2: data, API helpers, page objects and fixtures

IN REAL TEAMS Grow the framework when the tests ask for it. A layer nobody needs yet is code to maintain. The best frameworks are the ones where a new test takes ten lines and needs no new helper.

Lesson: How the framework grows

7.3 CI/CD pipelines

The framework ran with CI settings, a forgotten test.only failed the run, a wrong secret failed the sign in tests, shards merged into one report, and the smoke tests passed inside the official Docker images in all four languages.

KEY IDEAS

- **Every push** Tests run by themselves. Nobody has to remember.
- **Fresh machine** Install everything each time. Or use the Docker image.
- **Secrets** From the CI settings. Masked in the logs.
- **Evidence** Reports kept, pass or fail. Traces explain failures.
- **Remember** CI runs the same three steps everywhere: a machine that can run browsers, the packages and browsers installed, then the tests with CI settings and secrets.

CODE

Run the suite the way the pipeline runs it

playwright.config.js

```
import 'dotenv/config';
import { defineConfig, devices } from '@playwright/test';
import { env } from './src/config/env.js';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,                // a forgotten test.only fails the CI run
  retries: process.env.CI ? 2 : 0,
```

```
  workers: process.env.CI ? 2 : undefined,
  reporter: process.env.CI
    ? [['list'], ['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }]]
    : [['list'], ['html', { open: 'never' }]],
  use: {
    baseURL: env.baseURL,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    // { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    // { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

Run it: `CI=true npx playwright test && ls results`

Lesson: Running the suite the way the pipeline does

The workflow file each project keeps in .github/workflows

`.github/workflows/playwright.yml`

```
name: Playwright tests
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 14
```

Run it: `git push`

Lesson: GitHub Actions, line by line

What a run looks like when a secret is wrong

`src/data/users.js`

```
// Test accounts. Real projects read passwords from CI secrets, never from the repository.
export const users = {
  asha: { email: 'asha@shop.example', password: process.env.ASHA_PASSWORD || 's3cret', name: 'Asha' },
  ravi: { email: 'ravi@shop.example', password: process.env.RAVI_PASSWORD || 'admin123', name: 'Ravi' },
};
```

Run it: `ASHA_PASSWORD=wrong npx playwright test tests/smoke --reporter=line`

Lesson: Secrets and settings per environment

Each matrix job runs its own part of the suite

`.github/workflows/playwright.yml`

```
name: Playwright tests (sharded)
on:
  push:
```

```yaml
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false                # let every shard finish, even if one fails
      matrix:
        shardIndex: [1, 2]
        shardTotal: [2]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }} --reporter=blob
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
          name: blob-report-${{ matrix.shardIndex }}
          path: blob-report
          retention-days: 1

  merge-reports:
    if: ${{ !cancelled() }}           # merge even when a shard failed
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22
      - run: npm ci
      - uses: actions/download-artifact@v5
        with:
          path: all-blob-reports
          pattern: blob-report-*
          merge-multiple: true
      - run: npx playwright merge-reports --reporter html ./all-blob-reports
      - uses: actions/upload-artifact@v5
        with:
          name: html-report--attempt-${{ github.run_attempt }}
          path: playwright-report
          retention-days: 14
```

Run it: `for i in 1 2; do echo "== job shard $i of 2"; npx playwright test --shard=$i/2 --reporter=blob; mkdir -p all-blob-reports; mv blob-report/*.zip all-blob-reports/; done; echo "== job merge-reports"; npx playwright merge-reports --reporter=list ./all-blob-reports`

Lesson: Shards with a matrix, and one merged report

The framework inside the official Playwright image

package.json

```json
{
  "name": "shop-tests",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "playwright test",
    "test:smoke": "playwright test --grep @smoke",
    "report": "playwright show-report"
  },
  "devDependencies": {
    "@playwright/test": "1.63.0",
    "dotenv": "18.0.9"
```

```
  }
}
```

Run it: `docker run --rm --init --ipc=host --network host -v "$PWD":/work -w /work mcr.microsoft.com/playwright:v1.63.0-noble /bin/bash -c "npm ci && npx playwright test --grep @smoke"`

Lesson: The official Docker images

A Jenkinsfile for each project

Jenkinsfile

```
pipeline {
  agent { docker { image 'mcr.microsoft.com/playwright:v1.63.0-noble' } }
  environment {
    CI = 'true'                                  // Jenkins does not set CI by itself
    BASE_URL = 'https://staging.shop.example'
    ASHA_PASSWORD = credentials('asha-password')  // a Secret text credential in Jenkins
  }
  stages {
    stage('Install') {
      steps { sh 'npm ci' }
    }
    stage('Test') {
      steps { sh 'npx playwright test' }
    }
  }
  post {
    always {
      junit 'results/junit.xml'
      archiveArtifacts artifacts: 'playwright-report/**', allowEmptyArchive: true
    }
  }
}
```

Lesson: Jenkins, GitLab and Azure DevOps

TABLES

Table: on your laptop · on ci

	On your laptop	On CI
the machine	yours, with everything already installed	a fresh machine for every run
the browser	installed once	installed in every run, or already in a Docker image
the settings	a local .env file	variables and secrets in the CI settings
a failure	you watch it happen	you read the report, the logs and the trace afterwards
test.only	handy while debugging	must fail the run

Lesson: What CI does for your tests

Table: setting on ci · why

Setting on CI	Why
forbidOnly: true	a forgotten test.only stops the run instead of skipping the rest
retries: 2	a test that fails once and then passes is reported as flaky, not hidden
a fixed number of workers	CI machines are smaller than laptops. The Playwright docs suggest 1 for the most stable runs, or sharding for speed
trace: on-first-retry	every retried test leaves a trace to open later

Lesson: Running the suite the way the pipeline does

Table: line · what it does

Line	What it does
on: push / pull_request	when the workflow runs
runs-on: ubuntu-latest	a fresh Linux machine from GitHub
timeout-minutes: 30	stops a job that hangs. Keep it above any global timeout in the test config
actions/checkout	gets the code of the commit
setup-node, setup-python, setup-java	installs the language version
npm ci, pip install -r, mvn	installs the exact package versions from the lock or requirements file
playwright install --with-deps chromium	downloads Chromium and installs the Linux libraries it needs
env: with secrets. and vars.	hands settings and passwords to the tests
if: ${{ !cancelled() }}	uploads the results even when tests failed, which is when you need them most
upload-artifact	keeps the report as a download on the run page

Lesson: GitHub Actions, line by line

Table: trigger · use it for

Trigger	Use it for
push to main	checking what was merged
pull_request	checking a change before it is merged
schedule with cron	a nightly run of the full suite, for example in every browser
workflow_dispatch	a Run workflow button for runs by hand
deployment_status	testing a preview or staging deployment once it is ready

Lesson: GitHub Actions, line by line

Table: kind · in github · in the workflow · seen in logs

Kind	In GitHub	In the workflow	Seen in logs
a setting such as BASE_URL	Settings, Secrets and variables, Actions, Variables	${{ vars.BASE_URL }}	yes
a password	the same page, Secrets	${{ secrets.ASHA_PASSWORD }}	no, GitHub masks it as ***

Lesson: Secrets and settings per environment

Table: playwright test

	Playwright Test
split the suite	--shard=1/2
balance	by test count, with fullyParallel
one report	blob reports, then merge-reports

Lesson: Shards with a matrix, and one merged report

Table: language · image

Language	Image
JavaScript and TypeScript	mcr.microsoft.com/playwright:v1.63.0-noble

Lesson: The official Docker images

Table: flag · why the playwright docs recommend it

Flag	Why the Playwright docs recommend it
--ipc=host	without it Chromium can run out of memory and crash
--init	avoids special treatment of the process with id 1, a common cause of leftover processes
a fixed tag such as v1.63.0-noble	the image must match the Playwright version in the project. noble means Ubuntu 24.04
--user pwuser with a seccomp profile	only when the tests visit sites you do not trust

Lesson: The official Docker images

Table: ci service · file · sets ci=true · secrets

CI service	File	Sets CI=true	Secrets
GitHub Actions	.github/workflows/*.yml	yes	secrets.NAME
GitLab CI	.gitlab-ci.yml	yes	CI/CD variables, marked masked
Jenkins	Jenkinsfile	no, set it yourself	credentials('id')
Azure Pipelines	azure-pipelines.yml	no, set it yourself	secret pipeline variables, $(NAME)

Lesson: Jenkins, GitLab and Azure DevOps

Table: step · playwright test

Step	Playwright Test
1. read the failure in the job log	the list reporter output
2. download the artifact	playwright-report
3. open the trace	npx playwright show-report, then the trace
4. run only the failures again	npx playwright test --last-failed

Lesson: When the pipeline goes red

TIPS AND TRAPS

REMEMBER The Playwright documentation sums CI up in three steps: make sure the machine can run browsers, install the packages and browsers, then run the tests. Everything in this chapter is a version of those three steps.

Lesson: What CI does for your tests

IN REAL TEAMS In the repository settings, a branch rule can make this workflow a required check. Then a pull request cannot be merged while its tests are red.

Lesson: GitHub Actions, line by line

COMMON MISTAKE Secrets are not given to workflows that run for pull requests from forks, so a fork cannot read your passwords. Tests that need them are skipped or fail there, which is expected.

Lesson: Secrets and settings per environment

REMEMBER fail-fast: false keeps the other shards running when one fails, so you see every failure in one run. The merge job uses if: !cancelled() for the same reason.

Lesson: Shards with a matrix, and one merged report

IN REAL TEAMS In GitHub Actions, the same image goes under `container: image:` in the job, with `options: --user 1001`, and the browser install step is left out.

Lesson: The official Docker images

COMMON MISTAKE Never fix a red pipeline by adding a skip or a longer sleep. Find out whether the app or the test is wrong. The trace from the debugging chapter shows the page at every step of the failed run.

Lesson: When the pipeline goes red

IN REAL TEAMS A test marked flaky in the report passed on a retry. Treat it as a bug to fix this week, not as a pass.

Lesson: When the pipeline goes red

7.4 Reporting labs

One out of date test failed in every lab, and each report showed it: JUnit XML and JSON counts, the Playwright HTML report, pytest-html and ExtentReports with screenshots, Allure Report in all four languages, a custom summary, and one HTML report merged from two shards.

KEY IDEAS

- **For you** list, dot, -v. Right now.
- **For CI** Counts and history. Every server reads it.
- **For testers** Error, screenshot, trace. Why it failed.
- **For everyone** Same report, any language. Trends and labels.
- **Remember** Write several reports from one run: a short one for the terminal, JUnit XML for CI, and an HTML report with evidence for people.

CODE

Lab 1: the reports a machine reads

tests/products/price.spec.js

```
import { test, expect } from '../../src/fixtures/index.js';
import { products } from '../../src/data/products.js';

// Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
test('the desk lamp costs ₹1,199', async ({ productsPage }) => {
  await productsPage.open();
  await expect(productsPage.card(products.deskLamp.name).price).toHaveText('₹1,199');
});
```

Run it: `PLAYWRIGHT_JSON_OUTPUT_NAME=results/results.json PLAYWRIGHT_JUNIT_OUTPUT_NAME=results/junit.xml npx playwright test --reporter=dot,json,junit; grep -o '<testsuites [^>]*>' results/junit.xml; node -p "JSON.stringify(require('./results/results.json').stats)"`

Lesson: Lab 1: built-in reporters and JUnit XML

Lab 2: an HTML report for people

playwright.config.js

```
import 'dotenv/config';
import { defineConfig, devices } from '@playwright/test';
import { env } from './src/config/env.js';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,                // a forgotten test.only fails the CI run
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: process.env.CI
    ? [['list'], ['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }]]
    : [['list'], ['html', { open: 'never' }]],
  use: {
    baseURL: env.baseURL,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    // { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    // { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

Run it: `npx playwright test --reporter=line,html; ls playwright-report`

Lesson: Lab 2: HTML reports with evidence

Lab 3: Allure Report, the same report for every language

tests/products/price.spec.js

```
import * as allure from 'allure-js-commons';
import { test, expect } from '../../src/fixtures/index.js';
import { products } from '../../src/data/products.js';

// Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
test('the desk lamp costs ₹1,199', async ({ productsPage }) => {
  await allure.feature('Prices');
  await allure.severity('critical');
  await test.step('open the products page', async () => {
    await productsPage.open();
  });
  await test.step('check the lamp price', async () => {
    await expect(productsPage.card(products.deskLamp.name).price).toHaveText('₹1,199');
  });
});
```

Run it: `npx playwright test --reporter=line,allure-playwright; npx allure generate allure-results && ls allure-report/index.html`

Lesson: Lab 3: Allure Report in all four languages

Lab 4: write your own reporter

reporters/summary-reporter.js

```
// One line per test, then a short summary that a team could also send to chat or a dashboard
export default class SummaryReporter {
  passed = 0;
  failed = [];

  onBegin(config, suite) {
    console.log(`Starting ${suite.allTests().length} tests on ${config.workers} workers`);
  }

  onTestEnd(test, result) {
    const seconds = (result.duration / 1000).toFixed(1);
    if (result.status === 'passed') {
      this.passed++;
      console.log(`PASS  ${test.title} (${seconds} s)`);
    } else {
      this.failed.push(test.title);
      console.log(`FAIL  ${test.title} (${seconds} s)`);
    }
  }

  onEnd(result) {
    console.log(`${this.passed} passed, ${this.failed.length} failed, run ${result.status}`);
    for (const title of this.failed) console.log(`  to look at: ${title}`);
  }
}
```

Run it: `npx playwright test --reporter=./reporters/summary-reporter.js; echo "exit code $?"`

Lesson: Lab 4: a custom reporter

TABLES

Table: reader · wants · report

Reader	Wants	Report
you, at the terminal	pass or fail, right now	list, line or dot in Playwright Test, -v or -q in pytest, the Surefire summary in Maven
the CI server	counts and history	JUnit XML, which every CI server reads
a tester fixing a failure	the error, the screenshot, the trace	the Playwright HTML report, pytest-html, Allure, ExtentReports
a manager	a dashboard: how many passed, trends, which features	Allure, ExtentReports
your own tools	data in a known shape	JSON, or a custom reporter

Lesson: What a test report is for

Table: playwright test reporter · what it gives

Playwright Test reporter	What it gives
list	one line per test, the default on a laptop
line	one line that updates, and failures in full
dot	one character per test, for long runs on CI
html	a web page with every test, its steps, screenshots, videos and traces
json	all results as JSON, for your own tools
junit	JUnit XML for CI servers
blob	raw results of one shard, merged later with merge-reports
github	error notes on the right lines of a GitHub pull request

Lesson: Lab 1: built-in reporters and JUnit XML

Table: evidence · playwright test

Evidence	Playwright Test
screenshot at failure	use: { screenshot: 'only-on-failure' }
video	video: 'retain-on-failure'
trace	trace: 'on-first-retry'
your own file	testInfo.attach()

Lesson: Lab 2: HTML reports with evidence

Table: language · adapter · writes results with

Language	Adapter	Writes results with
TypeScript and JavaScript	allure-playwright (npm)	--reporter=allure-playwright

Lesson: Lab 3: Allure Report in all four languages

Table: playwright test

	Playwright Test
at the start	onBegin(config, suite)
after each test	onTestEnd(test, result)
at the end	onEnd(result)
turn it on	--reporter=./my-reporter.ts or the config

Lesson: Lab 4: a custom reporter

Table: where · how

Where	How
GitHub Actions	upload the report folder as an artifact, or publish it to GitHub Pages
Jenkins	the junit step for counts and history, and the HTML Publisher or Allure plugin for the page
GitLab CI	artifacts: reports: junit, and the report folder as an artifact or GitLab Pages
Azure DevOps	PublishTestResults for JUnit XML, and PublishPipelineArtifact for the HTML

Lesson: Lab 5: merging and publishing reports in CI

Table: need · typescript and javascript

Need	TypeScript and JavaScript
CI counts	junit reporter
debugging a failure	the built-in HTML report with traces
one report for many languages and teams	Allure
something special	a custom reporter

Lesson: Which report should a team use?

TIPS AND TRAPS

REMEMBER The labs in this chapter all use the framework from the framework chapter, plus one test that fails on purpose: it still expects the lamp to cost ₹1,199 after a price change to ₹1,299. A report is only interesting when something fails.

Lesson: What a test report is for

IN REAL TEAMS The report is only as good as its evidence. A failure with a screenshot and a trace can often be understood in a minute, without running anything again.

Lesson: Lab 2: HTML reports with evidence

NEW IN PLAYWRIGHT Both read the same allure-results folder.

Lesson: Lab 3: Allure Report in all four languages

IN REAL TEAMS Keep the allure-results of earlier runs, or the history folder of the last report, and Allure shows trends and flaky tests across runs.

Lesson: Lab 3: Allure Report in all four languages

COMMON MISTAKE A reporter must not change whether the run passed. It only describes the run, so CI still sees exit code 1 when a test failed.

Lesson: Lab 4: a custom reporter

REMEMBER CI servers read many files at once, for example target/surefire-reports/*.xml in Jenkins, so they do not need merging first. Allure also reads several allure-results folders in one command.

Lesson: Lab 5: merging and publishing reports in CI

7.5 What's new, and AI test agents

A 1.56 feature ran in all four languages, the agent definitions were created for real in the TypeScript framework, and a checklist shows how to keep AI-written tests honest.

KEY IDEAS

- **Releases** New browsers each time. Read the notes first.
- **New APIs** In every language. Like consoleMessages.
- **Test Agents** Plan, generate, heal. For Playwright Test.

- **Playwright MCP** A browser for assistants. Any language.
- **Remember** Update on a branch and read the release notes. Let AI write tests, but review them like any other code, and make sure each one can fail.

CODE

New in 1.56: read console messages, errors and requests at any time

tests/collect.spec.js

```
import { test, expect } from '@playwright/test';

test('reads console messages and page errors after the fact', async ({ page }) => {
  await page.goto('/widgets');
  await page.getByRole('button', { name: 'Write to the console' }).click();
  await Promise.all([
    page.waitForEvent('pageerror'),
    page.getByRole('button', { name: 'Break the page' }).click(),
  ]);

  // New in 1.56: the page keeps its recent console messages, errors and requests, no listener needed
  for (const m of await page.consoleMessages()) console.log(`console.${m.type()}: ${m.text()}`);
  for (const e of await page.pageErrors()) console.log(`page error: ${e.message}`);
  console.log(`requests so far: ${(await page.requests()).length}`);
  expect(await page.pageErrors()).toHaveLength(1);
});
```

Run it: `npx playwright test`

Lesson: A new feature in all four languages

TABLES

Table: version · highlights for testers

Version	Highlights for testers
1.56	Playwright Test Agents: planner, generator and healer. page.consoleMessages(), page.pageErrors() and page.requests()
1.57	a Speedboard tab in the HTML report that shows the slowest tests, and Chrome for Testing builds instead of Chromium
1.58	a timeline chart for merged reports, and old selectors such as _react removed
1.59	page.screencast for videos with chapters and overlays, page.ariaSnapshot(), and npx playwright trace on the command line
1.60	test.abort(), locator.drop() and HAR recording through tracing
1.61	passkeys through browserContext.credentials, page.localStorage, and -G as a short form of --grep-invert
1.62	retryStrategy, WebP screenshots, and npx playwright mcp bundled with Playwright
1.63	test locks, locator.visible(), --add-reporter and a perfetto reporter
1.64	page.webmcp, --shuffle, locator.within() and page.getByRef()

Lesson: How Playwright changes, and how to keep up

Table: language · update the package · then

Language	Update the package	Then
TypeScript and JavaScript	npm install -D @playwright/test@latest	npx playwright install --with-deps

Lesson: How Playwright changes, and how to keep up

Table: agent · reads · writes

Agent	Reads	Writes
planner	your running app, starting from the seed test	a Markdown plan in specs/
generator	a plan from specs/	test files in tests/, with each locator and check tried live
healer	a failing test	a fixed test, or a skipped one when it believes the app itself is broken

Lesson: Playwright Test Agents

Table: tool · use it for

Tool	Use it for
Playwright MCP	any assistant that speaks MCP, exploring an app and writing tests in any language
Playwright Test Agents	planning, generating and healing Playwright Test files in TypeScript or JavaScript
Copy prompt in the HTML report, trace viewer and UI mode	pasting a failed test with its error and page snapshot into any assistant, since 1.51
the Playwright CLI with skills	coding agents. The MCP README notes that the CLI uses fewer tokens

Lesson: Playwright MCP: a browser for AI assistants

Table: check · why

Check	Why
the test uses the framework: page objects, fixtures, test data	otherwise every generated test adds locators that someone must maintain
every check is meaningful	a test that only checks that a heading is visible passes even when the feature is broken
no fixed waits and no force: true	they hide timing problems that real users meet
the healer did not hide a bug	it may skip a test when the app looks broken. A skipped test needs a person to decide
the test fails when the feature breaks	break the feature on purpose once, or change an expected value, and watch it go red
secrets stay out of prompts and plans	plans and prompts are files and messages that other people and tools can read

Lesson: Using AI-written tests safely

TIPS AND TRAPS

IN REAL TEAMS Read the release notes before updating, and update on a branch so CI shows what changed. Each release lists its breaking changes, such as page.accessibility removed in 1.57 and Node.js 16 support removed in 1.54.

Lesson: How Playwright changes, and how to keep up

IN REAL TEAMS Use pageErrors() at the end of a test to fail it when the page threw an error that the user never saw. It is a cheap check that catches broken scripts early.

Lesson: A new feature in all four languages

COMMON MISTAKE The agents write Playwright Test code in TypeScript or JavaScript.

Lesson: Playwright Test Agents

IN REAL TEAMS Give the agents a seed test that signs in through your own fixtures. Generated tests then start from the same state as the rest of your suite.

Lesson: Using AI-written tests safely